

University of Agder

ENE418 — Data Analysis and Modelling Techniques in Renewable Energy

Project Report 03

Numerical Solution of 2D Ground Heat Transfer Around Boreholes Using
Finite Difference Methods

Submitted by

Bandhon Saha Bijoy (235053)
Joyonta Das Joy (235054)
Md Mehedy Hasan Sabby (235055)

Group: 03

Semester: Spring 2026

1 Introduction

This project investigates two-dimensional ground heat transport around boreholes using finite-difference numerical simulations. The study considers thermal diffusion, groundwater advection, combined advection–diffusion transport, time-varying heat injection, and multiple-borehole interaction.

2 Simulation Setup and Numerical Method

The simulations were performed using the **Group 3** configuration from Table 2 of the assignment. The groundwater velocity field is:

$$v_x = -0.02(y - 40) \quad (1)$$

$$v_y = 0.02(x - 40) \quad (2)$$

which produces a rotational groundwater circulation centred approximately around (40, 40) m.

For Tasks 4–5, the time-varying source term is:

$$Q(t) = \begin{cases} 0.15, & \text{mod}(t/1500) < 200 \\ 0, & \text{mod}(t/1500) \geq 200 \end{cases} \quad (3)$$

The computational domain size was 60×60 m with a base grid resolution of 200×200 . The initial ground temperature was $T_0 = 10^\circ\text{C}$ and the thermal diffusivity was $\alpha = 5 \times 10^{-3} \text{ m}^2/\text{s}$. The default time step used throughout the simulations was $\Delta t = 0.1$ s unless otherwise stated.

A finite-difference method (FDM) was used to discretise the governing equations on a structured Cartesian grid. Diffusion terms were solved using an explicit central-difference scheme, while advection terms were solved using an explicit upwind scheme for improved numerical stability. No-flux boundary conditions were applied along all domain boundaries.

3 Task 1 — Pure Diffusion

In Task 1, the two-dimensional pure diffusion equation was solved for a single borehole using a constant source term. Groundwater advection was neglected, so heat transport occurred only through thermal diffusion:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) + Q. \quad (4)$$

3.1 Temperature Field at Different Times

Figure 1 shows the temperature field for the pure diffusion case at $t = 2000$ s, $t = 5000$ s, and $t = 10000$ s.

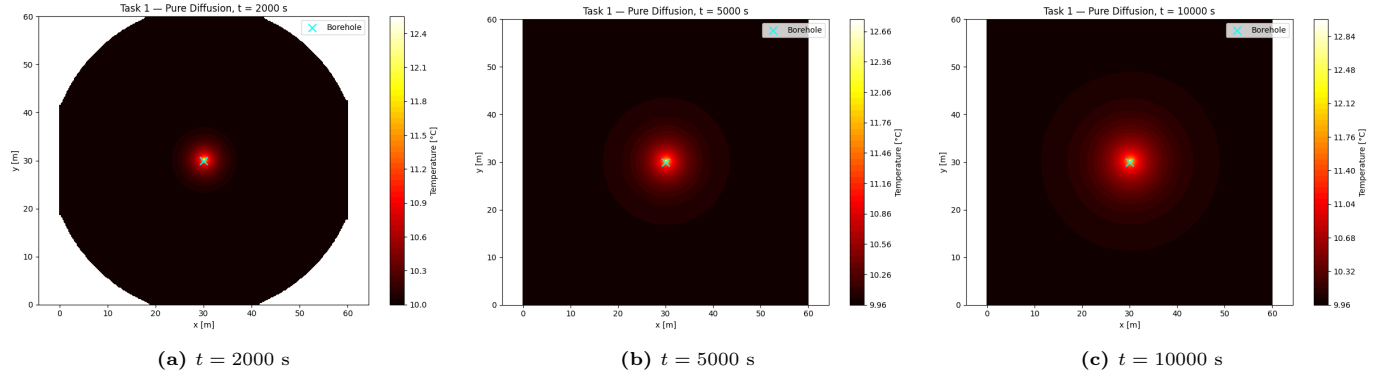


Figure 1: Temperature field for pure diffusion at different simulation times.

The plume spreads symmetrically outward from the borehole at (30, 30) m. Since no groundwater advection is included, heat is transported equally in all directions. As time increases, the plume radius increases and the temperature field becomes smoother.

3.2 Comparison with Analytical Diffusion Length

The analytical diffusion length was calculated using

$$L(t) = \sqrt{4\alpha t}. \quad (5)$$

Using $\alpha = 5 \times 10^{-3} \text{ m}^2/\text{s}$, the analytical diffusion lengths are summarized in Table 1.

Table 1: Analytical diffusion length for Task 1.

Time, t [s]	Diffusion length, $L(t)$ [m]
2000	6.32
5000	10.00
10000	14.14

The analytical result shows that the plume radius increases with time according to a square-root relationship. This agrees with the simulation results, where the plume grows outward while maintaining an approximately circular shape.

3.3 Effect of Increasing Thermal Diffusivity

To analyse the effect of thermal diffusivity, the pure diffusion simulation was repeated using

$$\alpha = 5 \times 10^{-3}, \quad 1 \times 10^{-2}, \quad 2 \times 10^{-2} \text{ m}^2/\text{s}.$$

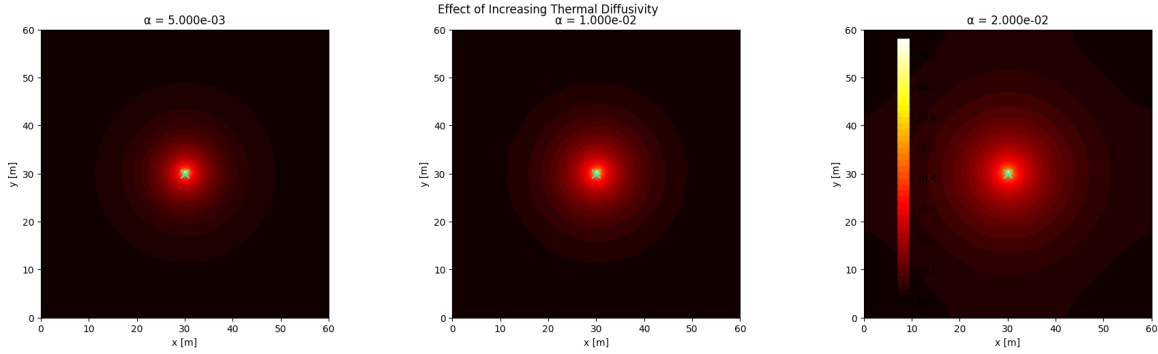


Figure 2: Effect of increasing thermal diffusivity on the temperature plume.

Figure 2 shows that increasing α causes heat to spread faster through the domain. A larger diffusivity produces a wider and smoother plume, while a smaller diffusivity keeps heat concentrated close to the borehole. Therefore, thermal diffusivity mainly controls the plume spreading rate.

3.4 Effect of Increasing or Decreasing Source Strength

The effect of source strength was investigated using

$$Q = 0.1, \quad 0.2, \quad 0.4 \text{ } ^\circ\text{C/s},$$

where $Q = 0.2^\circ\text{C/s}$ is the reference case.

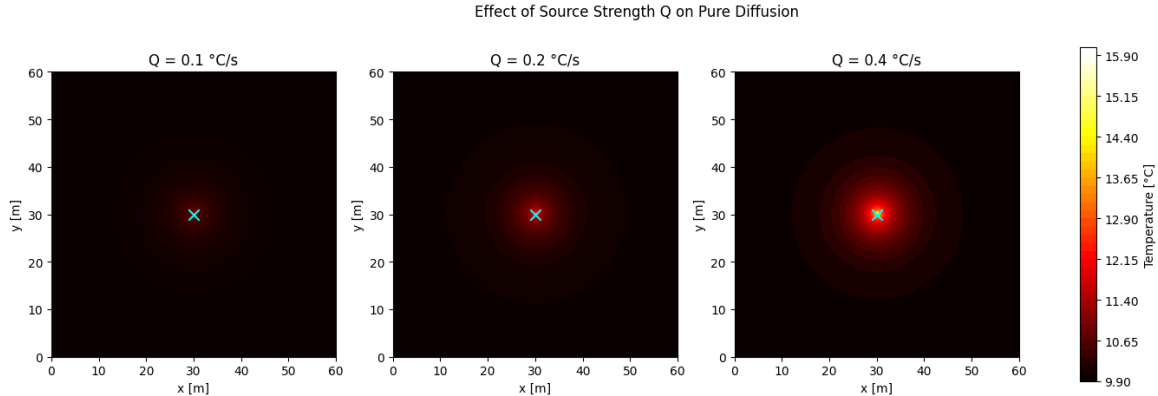


Figure 3: Effect of source strength on the thermal plume for pure diffusion.

Figure 3 shows that increasing Q produces a stronger thermal disturbance around the borehole. A larger source strength increases the peak temperature and plume intensity, but the spreading rate remains mainly controlled by thermal diffusivity α .

For smaller Q , the temperature rise above T_0 is weaker and the plume becomes less visible. This behaviour is comparable to seasonal borehole operation, where heating demand may vary between winter and summer.

3.5 Bonus: Numerical Detection of Plume Radius

The plume radius was detected numerically by identifying the maximum radial distance where

$$T - T_0 \geq 0.01^\circ\text{C}.$$

The numerical radius was compared with the analytical diffusion length in Table 2.

Table 2: Comparison between numerical plume radius and analytical diffusion length.

Time [s]	Numerical Radius [m]	Analytical $L(t)$ [m]	Difference [m]
2000	9.70	6.32	3.37
5000	15.26	10.00	5.26
10000	21.51	14.14	7.37

The numerical plume radius increased from 9.70 m at 2000 s to 21.51 m at 10000 s. It is larger than the analytical diffusion length because the chosen threshold includes weak temperature disturbances far from the borehole. However, both methods show the same trend of increasing plume radius with time.

3.6 Bonus: Grid Refinement Study

A grid refinement study was performed using 100×100 , 200×200 , and 400×400 computational grids.

The corresponding Fourier stability numbers were $Fo_{100} = 0.00272$, $Fo_{200} = 0.01100$, and $Fo_{400} = 0.04422$. All simulations remained within the explicit diffusion stability limit. Increasing the grid resolution improved plume

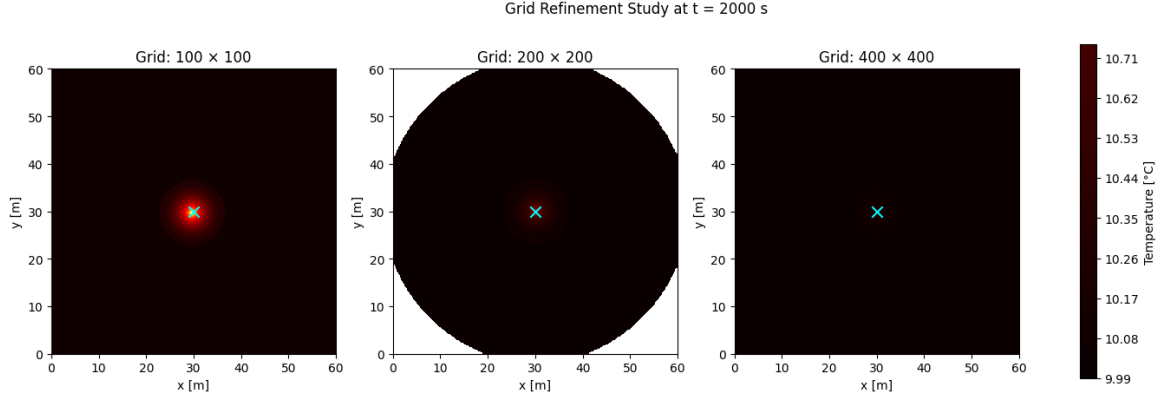


Figure 4: Grid refinement study for pure diffusion at $t = 2000$ s.

smoothness and reduced numerical anisotropy, with the 400×400 grid giving the closest approximation to the expected radially symmetric diffusion pattern.

3.7 Bonus: Time-Step Convergence Test

A time-step convergence study was performed using $\Delta t = 0.1, 0.05$, and 0.02 s. The simulations were compared at $t = 2000$ s.

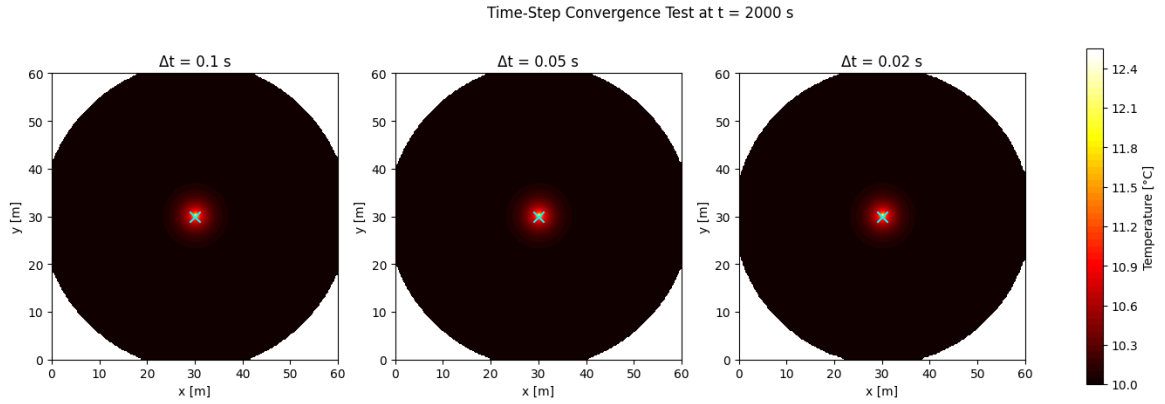


Figure 5: Time-step convergence test for pure diffusion at $t = 2000$ s.

The corresponding Fourier stability numbers were $Fo = 0.01111, 0.00556$, and 0.00222 , respectively.

Table 3: Time-step convergence error analysis.

Δt [s]	RMSE [$^{\circ}\text{C}$]	Maximum Error [$^{\circ}\text{C}$]
0.10	5.36×10^{-7}	5.74×10^{-6}
0.05	2.01×10^{-7}	2.15×10^{-6}
0.02	0.00	0.00

The temperature fields obtained using different time steps are nearly identical. The very small RMSE and maximum-error values confirm temporal convergence. Therefore, $\Delta t = 0.1$ s is sufficiently accurate and stable for the pure diffusion simulation.

4 Task 2 — Pure Advection

In Task 2, only groundwater-driven advection was considered. The diffusion term was neglected, so the governing equation becomes:

$$\frac{\partial T}{\partial t} + v_x \frac{\partial T}{\partial x} + v_y \frac{\partial T}{\partial y} = Q \quad (6)$$

For Group 3, the velocity field is:

$$v_x = -0.02(y - 40) \quad (7)$$

$$v_y = 0.02(x - 40) \quad (8)$$

This velocity field produces a rotational groundwater flow pattern centred approximately around $(40, 40)$.

4.1 Velocity Field

The Group 3 groundwater velocity field is shown in Figure 6.

Figure 6 shows that the groundwater velocity vectors rotate around the point $(40, 40)$. Since v_x depends on the y -coordinate and v_y depends on the x -coordinate, the flow direction changes continuously across the domain. Therefore, heat released from the borehole is expected to follow a curved trajectory rather than moving in a straight line.

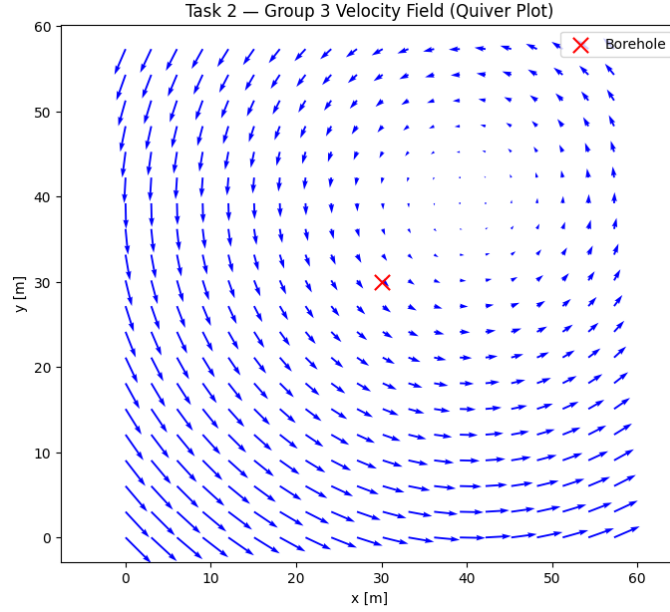


Figure 6: Group 3 groundwater velocity field shown using a quiver plot.

4.2 Plume Trajectory and Directional Behaviour

The resulting plume trajectory is shown in Figure 7.

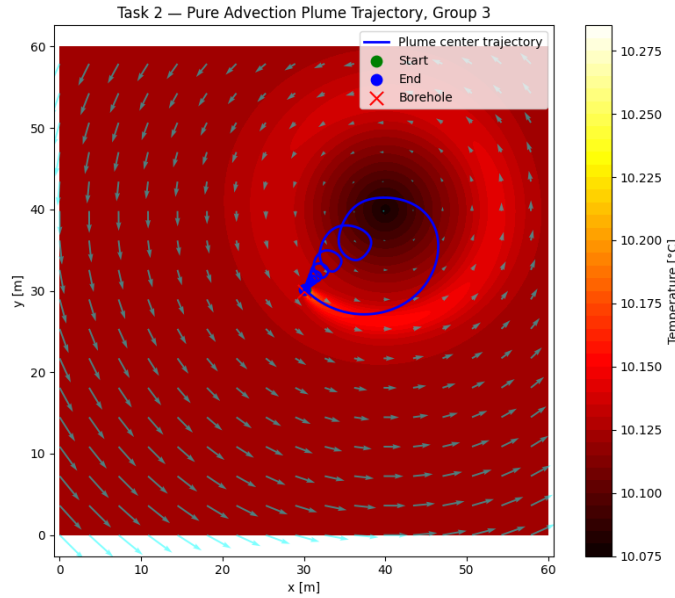


Figure 7: Pure advection plume trajectory for the Group 3 velocity field.

Figure 7 shows that the plume centre follows a curved, spiral-like trajectory around the rotational flow region. The borehole is located at approximately (30,30), and the local velocity vectors transport the heat toward the circulation region.

Unlike Task 1, where pure diffusion caused the plume to spread symmetrically in all directions, the pure advection solution is strongly direction-dependent. The plume motion is controlled by the local groundwater velocity field, so the plume becomes asymmetric and follows the rotational flow pattern.

This confirms that advection controls both the direction and movement of the thermal plume in Task 2.

4.3 Effect of Increasing Time Step

The effect of increasing the time step was analysed using:

$$\Delta t = 0.1, \quad 0.2, \quad 0.5 \text{ s}$$

The resulting temperature fields are shown in Figure 8.

The corresponding CFL numbers were:

- $\Delta t = 0.1 \text{ s} \rightarrow CFL = 0.53$
- $\Delta t = 0.2 \text{ s} \rightarrow CFL = 1.07$
- $\Delta t = 0.5 \text{ s} \rightarrow CFL = 2.67$

For $\Delta t = 0.1 \text{ s}$, the CFL number is approximately 0.53, so the solution remains stable and the plume follows the rotational velocity field. For $\Delta t = 0.2 \text{ s}$, the CFL number becomes slightly greater than 1. The plume shape still appears similar, but the solution is close to the stability limit and the numerical accuracy is reduced.

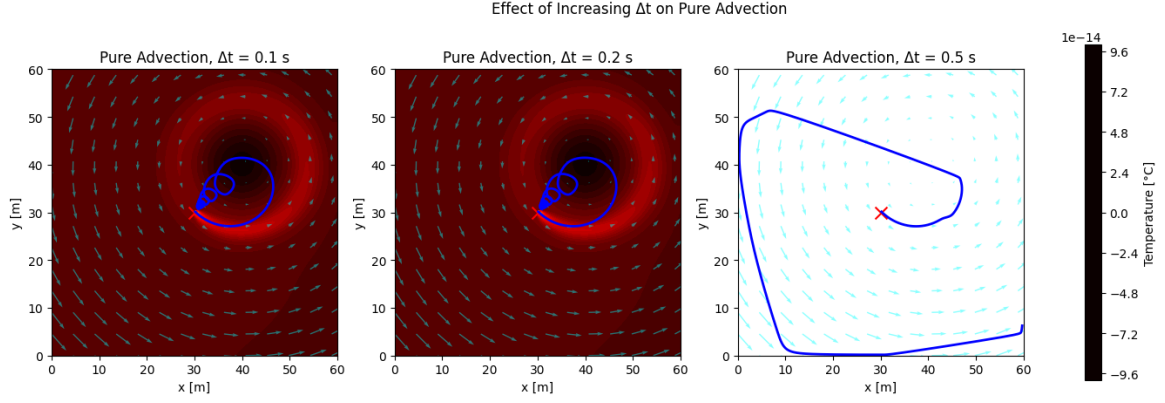


Figure 8: Effect of increasing time step on pure advection.

For $\Delta t = 0.5$ s, the CFL number is much larger than 1, causing numerical instability. This is visible from the distorted plume trajectory and unrealistic temperature field.

Compared with Task 1, increasing the time step has a stronger effect in the pure advection case. In pure diffusion, stability was mainly controlled by the Fourier number. In pure advection, stability is controlled by the CFL condition, which depends on velocity, grid spacing, and time step. Therefore, large time steps can quickly produce physically unrealistic plume movement.

4.4 Advection Scheme Comparison: Upwind vs Central Difference

The numerical behaviour of the upwind and central-difference advection schemes was compared for the pure advection problem.

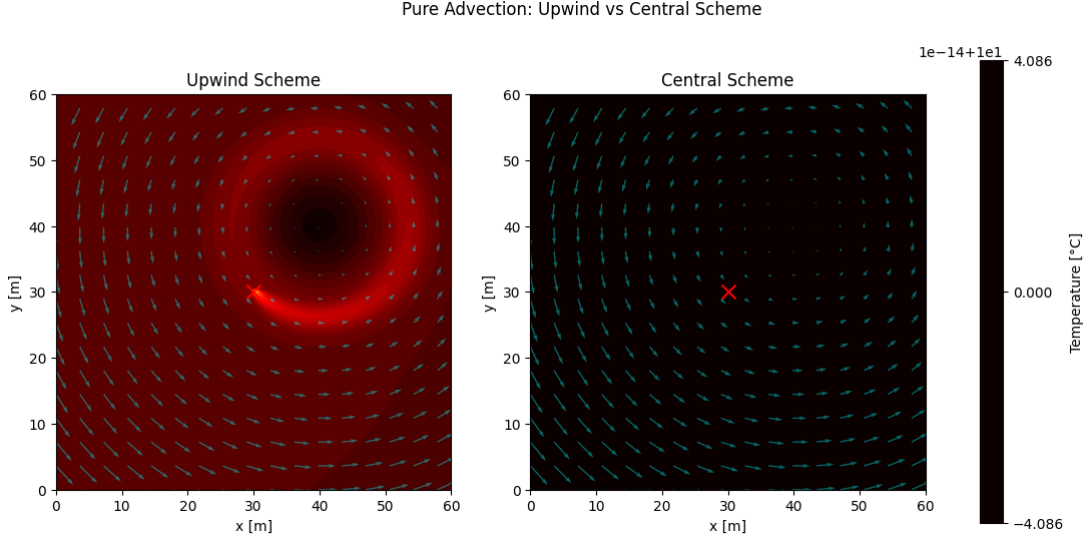


Figure 9: Comparison between upwind and central-difference advection schemes.

Figure 9 shows that the two schemes produce significantly different numerical behaviour.

The upwind scheme remains stable because it uses information from the upstream direction of the flow. This improves robustness for advection-dominated transport problems. However, the upwind method introduces numerical diffusion, causing the plume to appear smoother and slightly smeared.

The central-difference scheme reduces numerical diffusion and can preserve sharper plume structures. However, it is significantly less stable for explicit pure-advection simulations. In this case, the central scheme produced overflow warnings and nonphysical values, indicating numerical instability.

For the Group-3 rotational velocity field, the upwind scheme produced a smoother and more physically realistic plume trajectory, while the central scheme became unstable near steep temperature gradients.

Therefore, the upwind scheme is more suitable and reliable for explicit pure-advection simulations.

4.5 CFL Stability Analysis

The stability of the explicit advection solver is governed by the Courant–Friedrichs–Lewy (CFL) condition:

$$CFL = \frac{|v_x|\Delta t}{\Delta x} + \frac{|v_y|\Delta t}{\Delta y} \quad (9)$$

The maximum velocity magnitudes in the domain were:

$$|v_x|_{\max} = 0.8 \text{ m/s}$$

$$|v_y|_{\max} = 0.8 \text{ m/s}$$

Table 4 summarises the CFL analysis for different time steps.

Table 4: CFL stability analysis for pure advection.

Δt [s]	CFL_x	CFL_y	CFL_{total}	Stability
0.10	0.2667	0.2667	0.5333	Stable
0.20	0.5333	0.5333	1.0667	Near instability
0.50	1.3333	1.3333	2.6667	Unstable

The CFL analysis shows that the explicit upwind advection scheme remains stable only when the total CFL number is approximately below 1.

For $\Delta t = 0.1$ s, the CFL number remains below 1, so the plume trajectory is stable and accurately follows the rotational velocity field.

For $\Delta t = 0.2$ s, the CFL number slightly exceeds 1, meaning that the numerical transport per time step becomes too large relative to the grid spacing. The solution therefore approaches instability and numerical accuracy decreases.

For $\Delta t = 0.5$ s, the CFL number becomes much larger than 1. This produces numerical instability, overflow warnings, and unrealistic plume trajectories.

Compared with Task 1, the pure advection problem is significantly more sensitive to the time-step size because advection transport depends directly on local flow velocity. Therefore, maintaining a sufficiently small CFL number is essential for stable and physically realistic advection simulations.

4.6 Pure Advection Evolution at Different Times

The pure advection simulation results were also examined at:

$$t = 2000, \quad 5000, \quad 10000 \text{ s}$$

The resulting plume evolution is shown in Figure 10.

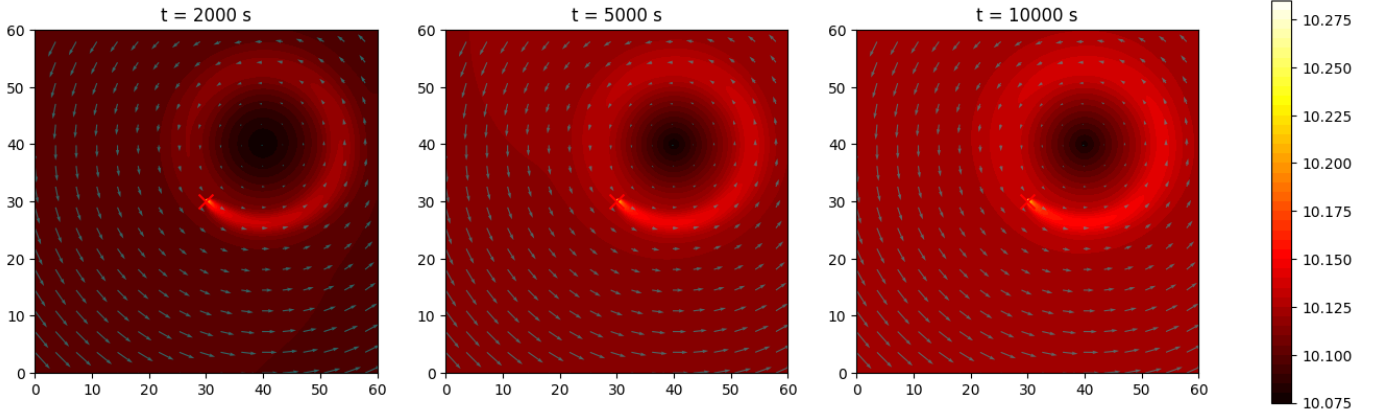


Figure 10: Pure advection plume evolution at different times.

Figure 10 shows that the plume continuously follows the rotational groundwater circulation over time. The plume remains relatively compact because diffusion is absent, while the advection field transports the heat around the vortex centre.

As time increases, the plume trajectory becomes longer and more curved, confirming that the groundwater velocity field dominates the thermal transport process in the pure advection case.

5 Task 3 — Advection and Diffusion

In Task 3, both groundwater advection and thermal diffusion were included. The governing equation becomes:

$$\frac{\partial T}{\partial t} + v_x \frac{\partial T}{\partial x} + v_y \frac{\partial T}{\partial y} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) + Q \quad (10)$$

This case combines the two main transport mechanisms studied separately in Task 1 and Task 2. Advection transports heat according to the groundwater velocity field, while diffusion spreads heat outward from regions of high temperature.

For Group 3, the velocity field is rotational:

$$v_x = -0.02(y - 40) \quad (11)$$

$$v_y = 0.02(x - 40) \quad (12)$$

A constant heat source was applied at the borehole located near (30, 30) m.

The diffusion stability numbers were $Fo_x = 0.00556$, $Fo_y = 0.00556$, and $Fo_{total} = 0.01111$. The corresponding advection stability numbers were $CFL_x = 0.2667$, $CFL_y = 0.2667$, and $CFL_{total} = 0.5333$. Since both the Fourier and CFL stability criteria are satisfied, the reference simulation with $\Delta t = 0.1$ s is numerically stable.

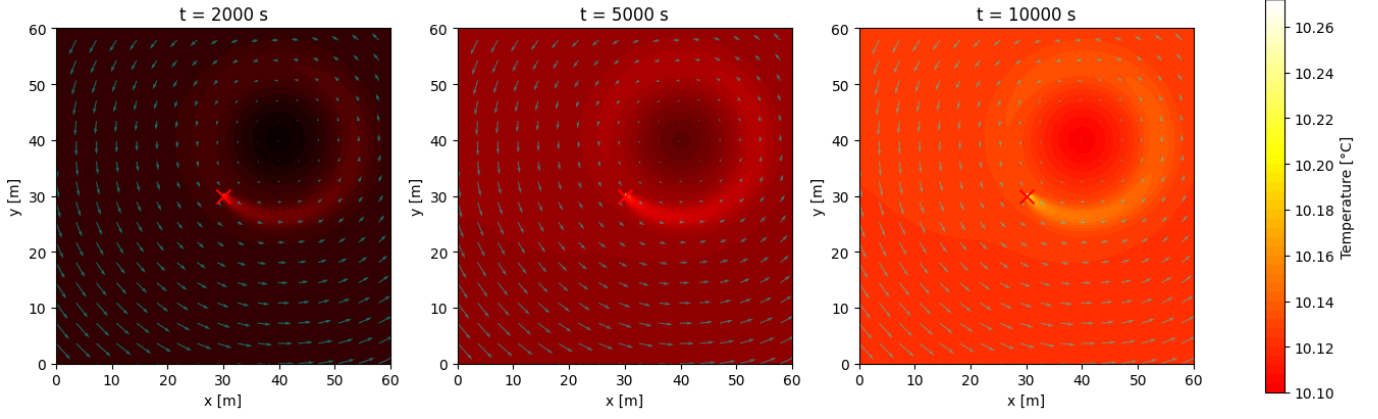


Figure 11: Temperature evolution for the advection–diffusion simulation.

5.1 Temperature Field Evolution

The temperature field was plotted at $t = 2000$ s, $t = 5000$ s, and $t = 10000$ s. The resulting temperature evolution is shown in Figure 11.

Figure 11 shows that the plume is affected by both rotational advection and thermal diffusion. The groundwater velocity field transports the plume around the circulation centre near (40, 40), while diffusion spreads heat outward and smooths the temperature gradients.

As time increases, the plume becomes broader and the temperature field expands over a larger region. However, unlike pure diffusion, the plume is not circular because advection introduces a preferred direction of transport.

5.2 Plume Trajectory

The trajectory of the plume centre is shown in Figure 12.

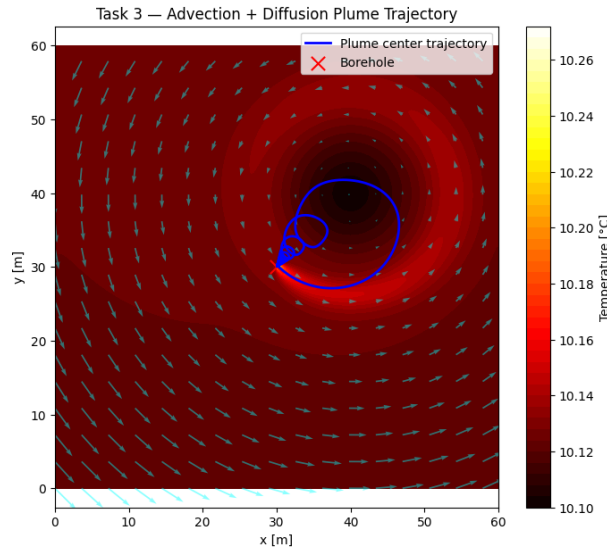


Figure 12: Plume centre trajectory for the advection–diffusion simulation.

Figure 12 shows that the plume centre follows a curved path around the rotational flow region. The trajectory is mainly controlled by the Group-3 velocity field. However, diffusion causes the plume to become wider and smoother compared with the pure advection case.

Therefore, the final plume behaviour is both directional and diffusive: advection controls plume movement, while diffusion controls plume spreading.

5.3 Comparison with Task 1 and Task 2

The results from Task 3 were compared with the pure diffusion and pure advection cases at $t = 10000$ s. The comparison is presented in Figure 13.

Figure 13 shows the different roles of diffusion and advection.

In Task 1, the plume is approximately circular and symmetric because diffusion spreads heat equally in all directions.

In Task 2, the plume becomes strongly direction-dependent because heat is transported by the Group-3 rotational velocity field. The plume follows a curved path around the vortex centre and does not spread radially.

In Task 3, both mechanisms act together. Advection controls the main direction of plume movement, while diffusion spreads and smooths the temperature field around the advective pathway. Therefore, the Task 3 plume is curved like the pure advection case but broader and smoother due to diffusion.

This demonstrates the competing roles of the two processes: advection transports heat directionally with groundwater flow, whereas diffusion reduces sharp gradients and spreads heat outward from high-temperature regions.

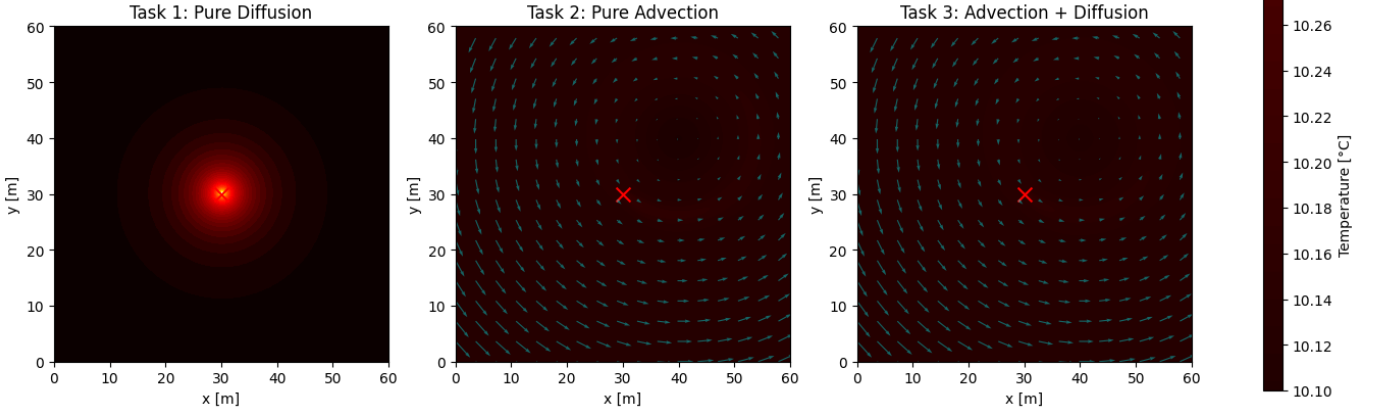


Figure 13: Comparison of pure diffusion, pure advection, and advection–diffusion at $t = 10000$ s.

5.4 Effect of Increasing Time Step

The time-step sensitivity of the advection–diffusion simulation was analysed using:

$$\Delta t = 0.1, \quad 0.2, \quad 0.5 \text{ s}$$

The resulting temperature fields are shown in Figure 14.

Effect of Increasing Δt on Advection-Diffusion

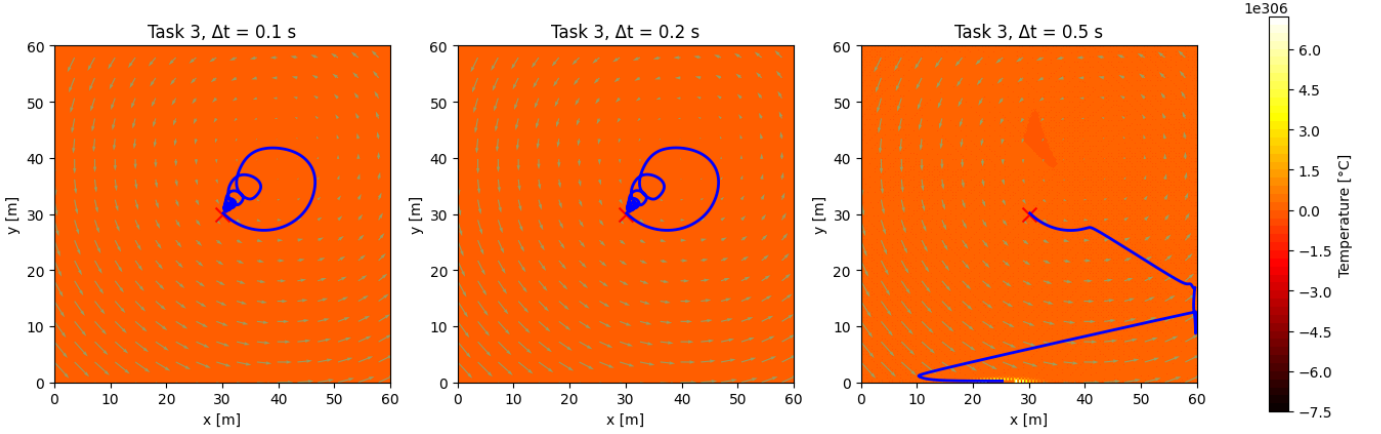


Figure 14: Effect of increasing time step on the advection–diffusion simulation.

The stability values are summarised in Table 5.

Table 5: Stability analysis for different time steps in Task 3.

Δt [s]	For_{total}	CFL_{total}	Status
0.1	0.01111	0.5333	Stable
0.2	0.02222	1.0667	Unstable / risky
0.5	0.05556	2.6667	Unstable

5.5 Bonus: Péclet Number Analysis

To quantify the relative importance of advection and diffusion, the Péclet number was evaluated using

$$Pe = \frac{vL}{\alpha} \quad (13)$$

where v is the characteristic groundwater velocity, L is the characteristic length scale, and α is the thermal diffusivity.

For the present simulation, the characteristic length was $L = 60$ m, while the mean and maximum velocity magnitudes were $v_{mean} = 0.5190$ m/s and $v_{max} = 1.1314$ m/s, respectively.

The resulting Péclet numbers are summarised in Table 6.

Table 6: Péclet number analysis for Task 3.

Velocity used	Velocity [m/s]	Péclet Number
Mean velocity	0.5190	6228.33
Maximum velocity	1.1314	13576.45

Both Péclet numbers are several orders of magnitude larger than unity, indicating that the system is strongly advection-dominated. This observation agrees with the numerical results, where the overall plume trajectory is governed primarily by the Group-3 rotational groundwater flow field. Diffusion remains important for smoothing temperature gradients and broadening the plume, but it does not control the main direction of thermal transport.

The large Péclet numbers therefore confirm that advective transport dominates heat movement throughout the computational domain.

5.6 Bonus: Thermal Mass and Energetics

To investigate energy conservation and thermal accumulation within the domain, the total thermal mass was calculated as

$$M(t) = \iint (T - T_0) dx dy \quad (14)$$

where T_0 denotes the initial ground temperature.

The thermal mass represents the total temperature excess stored within the computational domain relative to the initial condition. The evolution of thermal mass over time is shown in Figure 15.

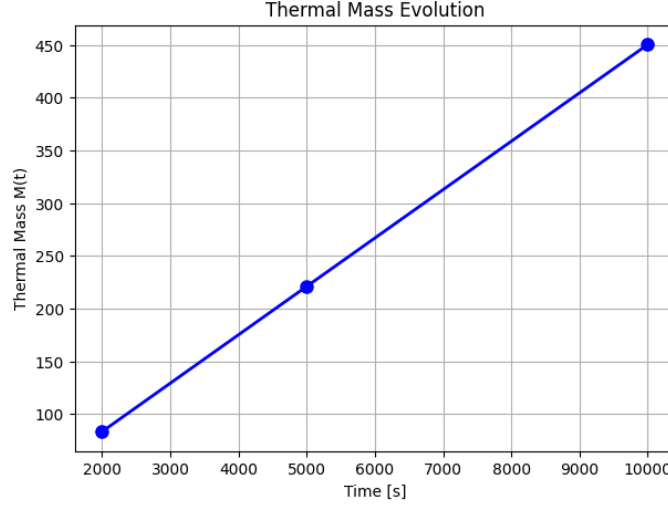


Figure 15: Thermal mass evolution for the advection–diffusion simulation.

The corresponding thermal-mass values are listed in Table 7.

Table 7: Thermal mass evolution in Task 3.

Time [s]	Thermal Mass $M(t)$
2000	83.27
5000	220.97
10000	450.32

The thermal mass increases continuously with time because the borehole acts as a constant heat source. As thermal energy is injected into the ground, advection redistributes this energy according to the groundwater velocity field, while diffusion spreads the heat spatially and smooths temperature gradients.

The relatively smooth increase in $M(t)$ suggests that the numerical advection–diffusion scheme preserves energy reasonably well, apart from normal discretisation errors associated with the finite-difference formulation.

Overall, the energetics analysis confirms that:

- thermal energy continuously accumulates because of the constant source term,
- advection transports thermal energy directionally along the groundwater flow field,
- diffusion smooths the temperature distribution and spreads heat spatially,
- the numerical model maintains physically consistent energy evolution throughout the simulation.

6 Task 4 — Time-Varying Source

In Task 4, the constant source term from Task 3 was replaced with a time-varying heat source $Q(t)$. The governing equation remains the full advection–diffusion equation:

$$\frac{\partial T}{\partial t} + v_x \frac{\partial T}{\partial x} + v_y \frac{\partial T}{\partial y} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) + Q(t) \quad (15)$$

For Group 3, the source term is pulse-like:

$$Q(t) = \begin{cases} 0.15, & \text{mod}(t/1500) < 200 \\ 0, & \text{mod}(t/1500) \geq 200 \end{cases} \quad (16)$$

This means that heat is injected only during the first 200 s of each 1500 s cycle.

6.1 Temporal Evolution of the Temperature Field

The behaviour of the time-varying source term is illustrated in Figure 16. The source alternates between active and inactive periods, injecting heat only during the first 200 s of every 1500 s cycle. During inactive periods, no additional heat is supplied; however, previously injected thermal energy continues to be transported by groundwater advection and spread through diffusion.

The resulting temperature field was examined at $t = 2000$ s, $t = 5000$ s, and $t = 10000$ s.

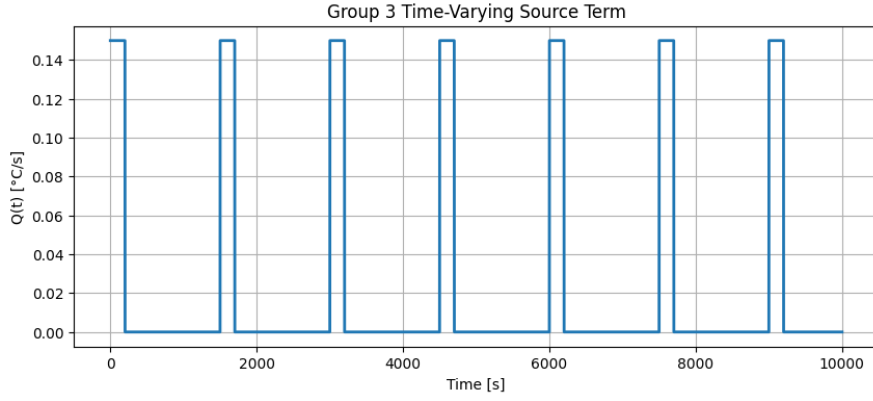


Figure 16: Group 3 time-varying heat source $Q(t)$.

Task 4 — Time-Varying Source Evolution

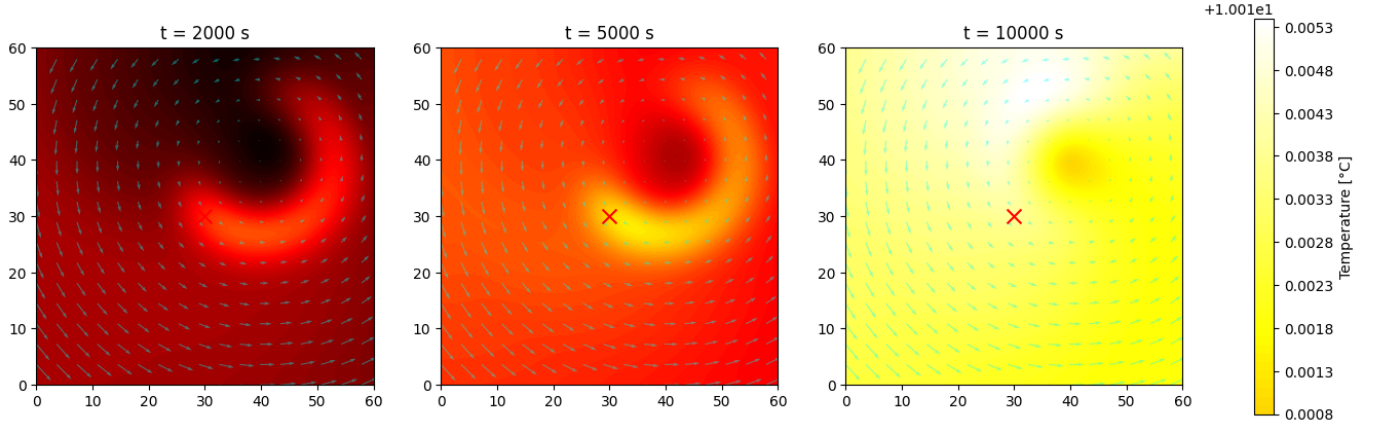


Figure 17: Temporal evolution of the temperature field under the time-varying source term.

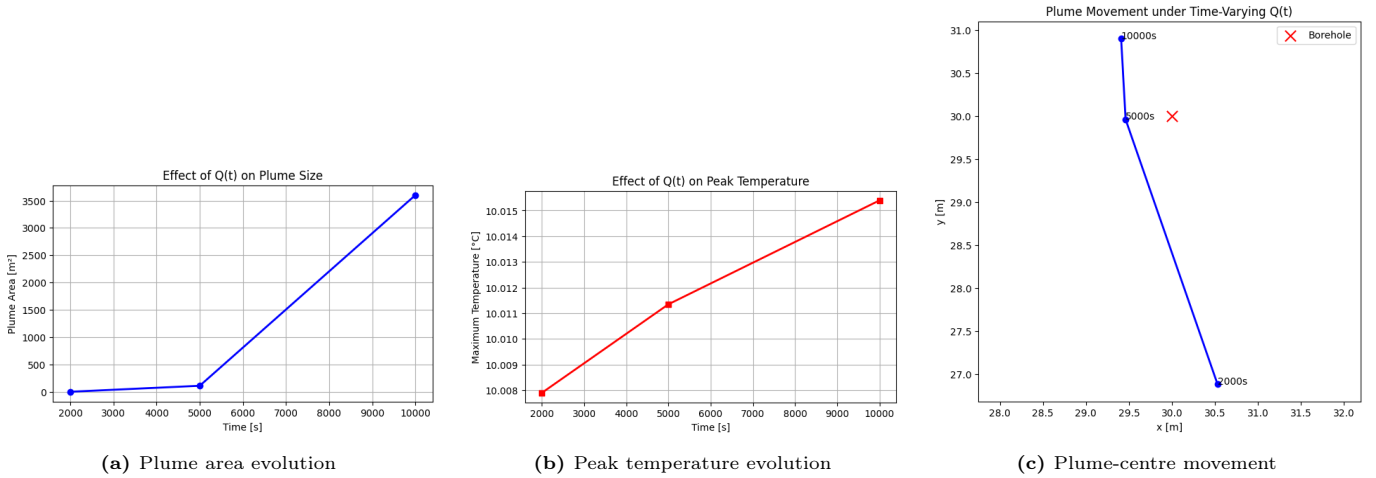


Figure 18: Effect of the time-varying source term on plume characteristics.

Figure 17 shows that the plume develops gradually because heat is injected intermittently rather than continuously. At early times, the thermal disturbance remains relatively weak because only a limited amount of heat has entered the domain. As additional source cycles occur, thermal energy accumulates and the plume expands.

The plume follows the Group 3 rotational groundwater velocity field, producing curved transport around the circulation region centred near (40, 40) m. At the same time, diffusion smooths the temperature gradients and broadens the thermal influence zone. Compared with the constant-source simulation in Task 3, the overall temperature increase is smaller because heat is supplied only during a fraction of the total simulation period.

6.2 Effect of $Q(t)$ on Plume Size, Shape, Direction, and Movement

The influence of the periodic source was analysed through plume area, peak temperature, and plume-centre displacement.

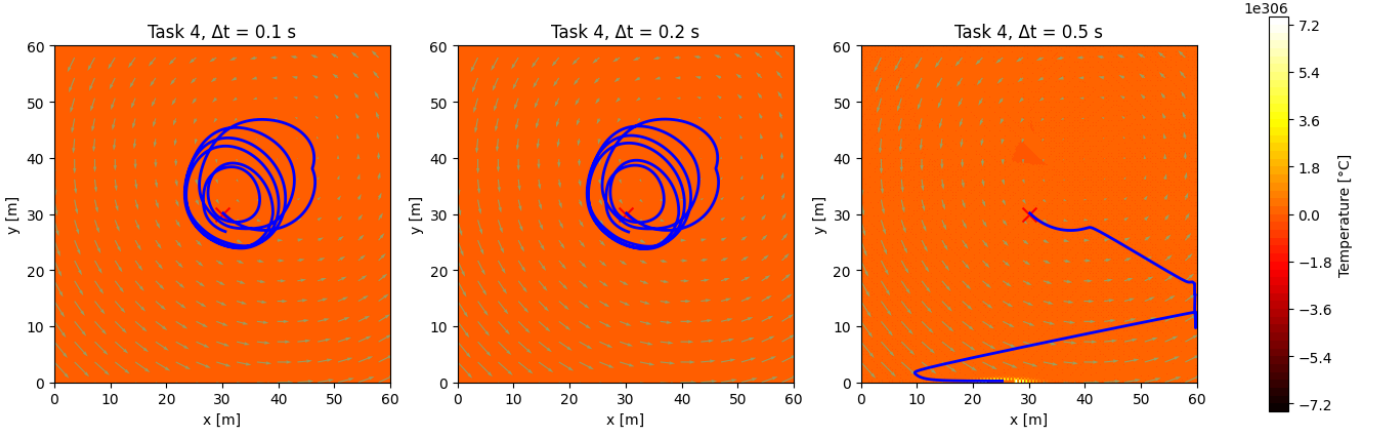
Figure 18 shows that the plume area increases with time because thermal energy accumulates over successive heating cycles. Although the source remains inactive for most of each cycle, previously injected heat continues to be transported by groundwater flow and spread by diffusion.

The peak temperature also increases gradually throughout the simulation. However, the increase remains moderate because heat injection occurs only intermittently. The plume shape remains asymmetric due to the rotational groundwater velocity field, while diffusion smooths the temperature gradients and broadens the plume.

The plume centre continuously shifts position because thermal energy is advected along a curved groundwater

Table 8: Summary of plume characteristics under the time-varying source term.

Time [s]	Area [m ²]	Peak T [°C]	x_c [m]	y_c [m]
2000	0.00	10.0079	30.53	26.89
5000	109.35	10.0113	29.46	29.96
10000	3600.00	10.0154	29.41	30.90

Effect of Increasing Δt on Time-Varying Source Simulation**Figure 19:** Effect of increasing time step on the time-varying source simulation.**Table 9:** Stability analysis for Task 4 using different time steps.

Δt [s]	Fo_{total}	CFL_{total}	Status
0.1	0.01111	0.5333	Stable
0.2	0.02222	1.0667	Unstable / Risky
0.5	0.05556	2.6667	Unstable

pathway. Since the local flow direction varies throughout the domain, plume movement is non-radial and follows the rotational circulation pattern.

The results demonstrate that the time-varying source primarily influences plume intensity and thermal energy accumulation. In contrast, the overall transport direction and plume trajectory remain controlled by the Group 3 groundwater velocity field.

6.3 Effect of Increasing Time Step and Comparison with Task 3

The sensitivity of the solution to the time-step size was investigated using

$$\Delta t = 0.1, \quad 0.2, \quad 0.5 \text{ s.}$$

For $\Delta t = 0.1$ s, both the Fourier and CFL stability conditions are satisfied. The simulation remains stable and accurately captures the periodic ON/OFF behaviour of the source.

For $\Delta t = 0.2$ s, the CFL number exceeds unity. Although the overall plume pattern remains similar, numerical accuracy decreases because the source cycles are represented using fewer computational time steps.

For $\Delta t = 0.5$ s, the CFL condition is strongly violated, resulting in numerical instability and unrealistic plume transport.

Compared with Task 3, the time-varying source problem is more sensitive to increasing time-step size because temporal discretisation affects both transport stability and source representation. Consequently, large time steps reduce numerical accuracy while also limiting the ability to resolve the periodic heating cycles. Therefore, $\Delta t = 0.1$ s provides the most reliable and physically realistic solution.

6.4 Bonus — Plume Centre Trajectory Analysis

To further analyse the influence of the time-varying source and the rotational groundwater velocity field, the trajectory of the plume centre was tracked continuously throughout the simulation.

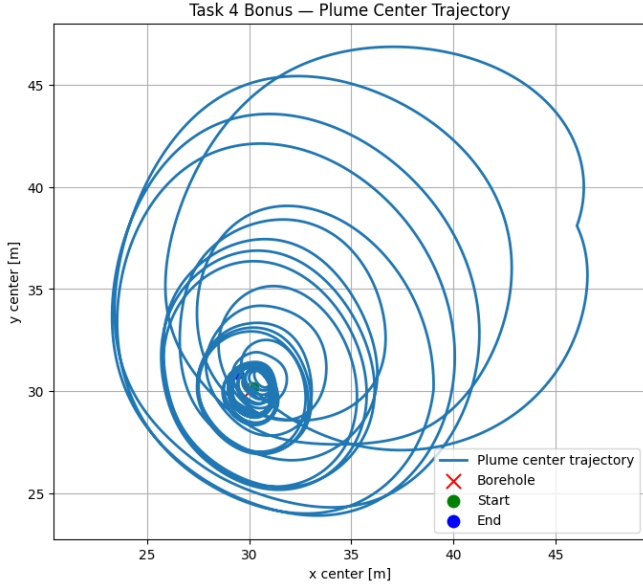
Figure 20 shows both the spatial trajectory of the plume centre and the temporal evolution of its coordinates. The plume centre does not remain fixed at the borehole location. Instead, it follows a curved spiral-like trajectory around the borehole region, confirming that groundwater advection dominates the large-scale plume movement.

The plume centre initially starts at approximately $(x_c, y_c) = (30.15, 30.15)$ m, which corresponds closely to the borehole location. At the end of the simulation, the plume centre shifts to approximately $(x_c, y_c) = (29.41, 30.90)$ m, showing that the plume has been transported by the rotational groundwater circulation.

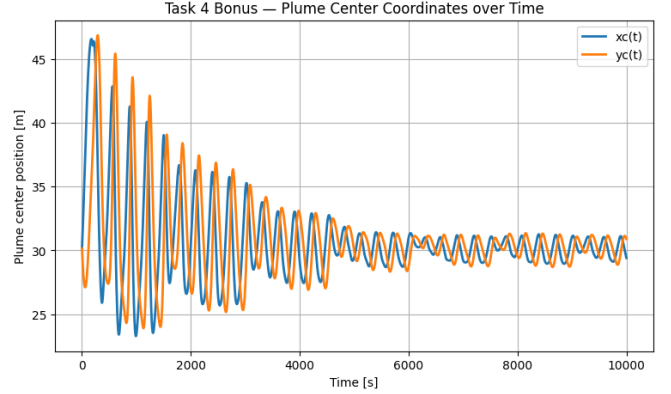
The coordinate histories reveal that both $x_c(t)$ and $y_c(t)$ oscillate with time because the plume continuously responds to the combined effects of rotational groundwater advection, thermal diffusion, and the periodic ON/OFF heat injection represented by $Q(t)$.

At early times, the oscillation amplitude is relatively large because newly injected heat is strongly affected by the local velocity field. As time increases, diffusion smooths the plume and reduces sharp thermal gradients, causing the trajectory to become more damped and concentrated near the borehole region.

The periodic behaviour also reflects the intermittent source term. During active heating periods, new thermal energy is injected and transported by groundwater flow, slightly shifting the plume centre. During inactive periods, diffusion



(a) Plume-centre trajectory



(b) Coordinates $x_c(t)$ and $y_c(t)$

Figure 20: Plume-centre movement under the Group-3 rotational groundwater velocity field.

Table 10: Borehole locations used in Task 5.

Borehole	x [m]	y [m]
B1	19.8	19.8
B2	39.9	19.8
B3	19.8	39.9
B4	39.9	39.9

and advection redistribute the existing thermal energy without additional heat input.

Overall, the plume-centre trajectory demonstrates that the Group-3 rotational velocity field controls the dominant transport direction, while the time-varying source term modulates plume intensity and temporal oscillation behaviour.

7 Task 5 — Multiple Boreholes and Thermal Interference

Task 5 extends the advection–diffusion model by introducing four boreholes instead of a single heat source. The objective is to investigate thermal plume interaction and interference between neighbouring boreholes under the Group-3 rotational groundwater velocity field.

The governing equation remains

$$\frac{\partial T}{\partial t} + v_x \frac{\partial T}{\partial x} + v_y \frac{\partial T}{\partial y} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) + Q \quad (17)$$

where advection transports heat through groundwater motion while diffusion spreads heat spatially.

7.1 Borehole Configuration

Four boreholes were positioned inside the computational domain as shown in Table 10.

The nearest numerical grid locations were:

- B1 $\rightarrow$ (19.90, 19.90) m
- B2 $\rightarrow$ (39.80, 19.90) m
- B3 $\rightarrow$ (19.90, 39.80) m
- B4 $\rightarrow$ (39.80, 39.80) m

7.2 Temperature Field Evolution and Thermal Interference

Figure 21 shows the temperature field at $t = 2000$ s, $t = 5000$ s, and $t = 10000$ s. Initially, four individual thermal plumes develop around the boreholes. As time increases, diffusion broadens the plumes while advection transports them according to the rotational groundwater flow field.

The thermal plumes gradually overlap, creating interference regions between neighbouring boreholes. The overlap becomes increasingly visible at later times, particularly in the upper-right part of the domain where groundwater circulation transports thermal energy.

The strongest temperature increase occurs near Borehole B4 because the Group-3 velocity field tends to transport heat toward the upper-right region. Consequently, plume interaction becomes more significant in this area compared with the lower part of the domain.

7.3 Borehole Interaction Analysis

To quantify thermal interference, midpoint temperatures between borehole pairs were analysed at $t = 10000$ s.

The interaction results are summarised in Table 11.

The strongest interaction occurred between Boreholes B3 and B4, which produced the highest midpoint temperature increase:

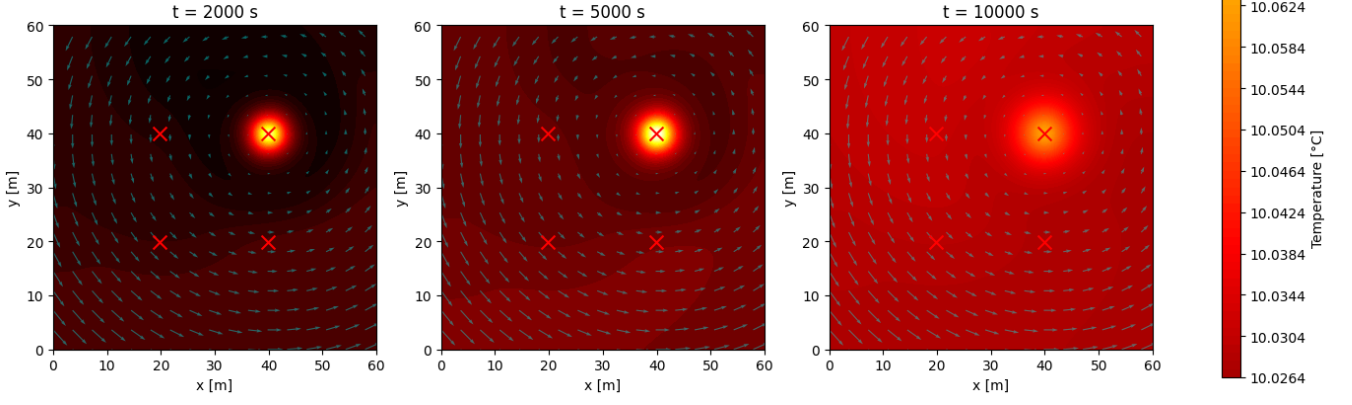
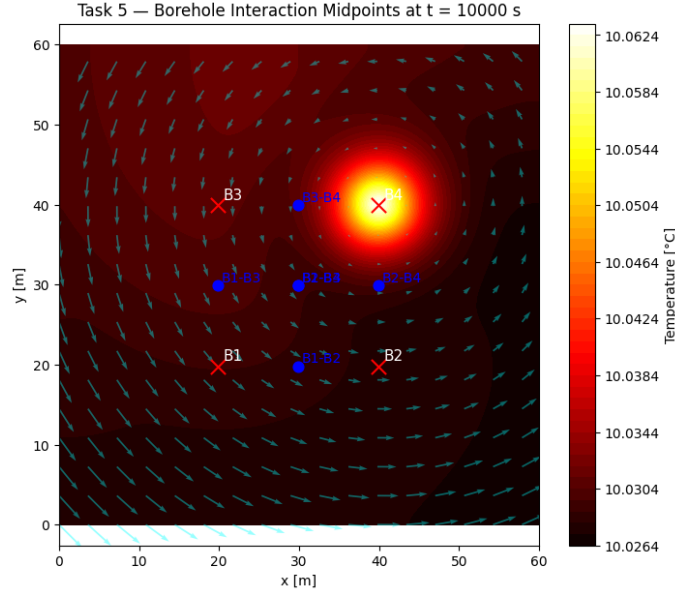

Figure 21: Temperature evolution for the four-borehole simulation at different times.

Figure 22: Midpoint interaction analysis between boreholes at $t = 10000$ s.

Table 11: Thermal interaction between borehole pairs at $t = 10000$ s.

Pair	Distance [m]	T_{mid} [°C]	ΔT_{mid} [°C]
B1–B2	20.10	10.02851	0.02851
B1–B3	20.10	10.02994	0.02994
B1–B4	28.43	10.02897	0.02897
B2–B3	28.43	10.02897	0.02897
B2–B4	20.10	10.02959	0.02959
B3–B4	20.10	10.03085	0.03085

$$\Delta T_{\text{mid}} = 0.03085^\circ\text{C}$$

This stronger interaction can be explained by three factors:

- relatively short spacing between the boreholes,
- favourable alignment with the rotational groundwater circulation,
- diffusion-induced broadening of neighbouring thermal plumes.

The Group-3 velocity field transports thermal energy preferentially toward the upper-right portion of the domain. As a result, thermal plumes originating from B3 and B4 overlap more strongly than other borehole pairs. This demonstrates how groundwater flow direction can significantly influence borehole interference in addition to geometric spacing.

7.4 Bonus — Superposition Test

To verify the linearity of the governing advection–diffusion equation, the multiple-borehole solution was compared with the sum of four independent single-borehole simulations.

Figure 23 compares the direct four-borehole simulation with the superposition solution. The left panel shows the temperature field obtained from the simultaneous four-borehole simulation, while the middle panel shows the temperature field obtained by summing four separate single-borehole simulations. The right panel displays the difference field between the two approaches.

The agreement between the two solutions was quantified using the root-mean-square error (RMSE) and the maximum absolute error:

$$RMSE = 4.90 \times 10^{-13}$$

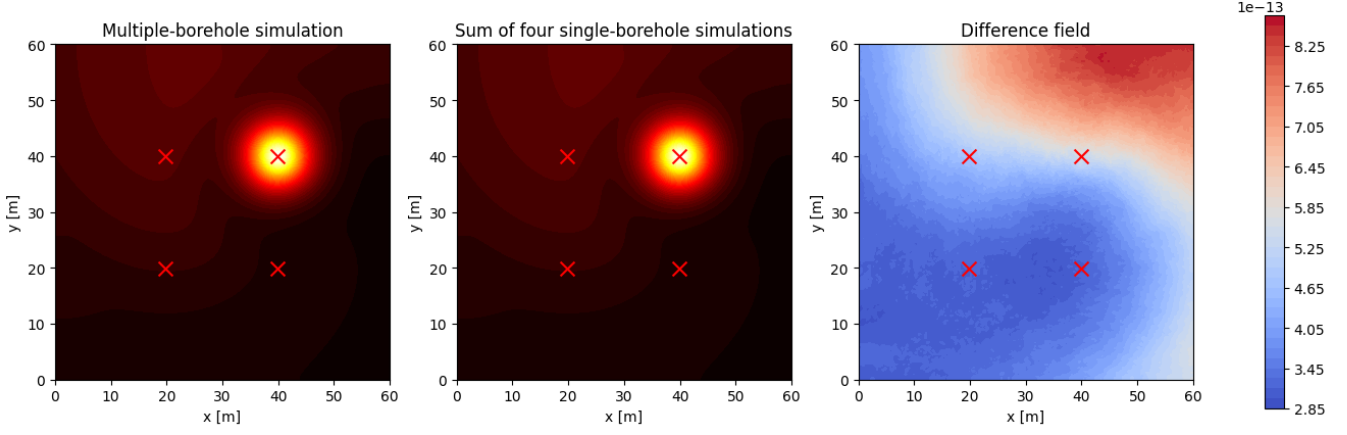


Figure 23: Comparison between the direct four-borehole simulation and the superposition of four independent single-borehole solutions.

$$\text{Maximum Error} = 8.60 \times 10^{-13}$$

These errors are several orders of magnitude smaller than the temperature variations present in the simulation and are therefore negligible.

The difference field remains extremely small throughout the computational domain. Both the RMSE and maximum absolute error are of the order of 10^{-13} , which corresponds to numerical round-off error associated with floating-point arithmetic.

Since these errors are effectively zero relative to the simulated temperature changes, the direct multi-borehole solution and the superposition solution are practically identical.

Therefore, the simulation confirms that the governing advection–diffusion equation behaves linearly under the conditions considered in this project. The thermal response generated by multiple boreholes can be accurately represented as the sum of the individual borehole responses, demonstrating excellent agreement with the superposition principle.

8 Discussion and Observations

The simulations demonstrated the different behaviours of thermal transport under diffusion, advection, and combined advection–diffusion processes.

In Task 1, pure diffusion produced a symmetric circular plume because heat spread equally in all directions. Increasing thermal diffusivity α increased plume spreading, while increasing source strength Q increased plume intensity and peak temperature. The numerical plume radius followed the same general trend as the analytical diffusion length $L(t) = \sqrt{4\alpha t}$.

In Task 2, groundwater advection dominated the transport behaviour. The Group-3 rotational velocity field caused the plume to follow a curved trajectory instead of spreading radially. The simulations also showed that pure advection is highly sensitive to the CFL stability condition. Large time steps produced numerical instability and unrealistic plume behaviour.

In Task 3, the combined advection–diffusion model produced a plume that was both directional and spatially spread. Advection controlled the main plume movement, while diffusion smoothed temperature gradients and broadened the plume. The Péclet number analysis confirmed that the system is strongly advection-dominated.

In Task 4, the time-varying source term introduced periodic thermal injection. The plume intensity increased during active source periods, while advection and diffusion continued to redistribute heat during inactive periods. The plume centre followed an oscillatory rotational trajectory due to the combined influence of the velocity field and the periodic source.

In Task 5, multiple boreholes produced plume overlap and thermal interference. The strongest interactions occurred between boreholes aligned favourably with the rotational groundwater circulation. The superposition test confirmed that the governing advection–diffusion system behaved approximately linearly under the simulated conditions.

Overall, the results demonstrated that diffusion mainly controls plume spreading, while advection controls plume direction and transport. The simulations also showed the importance of satisfying Fourier and CFL stability conditions for accurate and stable numerical solutions.

A Appendix

A.1 Task 1: Pure Diffusion Setup and Stability Check

Listing 1: Task 1: Pure diffusion full Python code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # =====
5 # PARAMETERS
6 # =====
7
8 Lx = 60.0
9 Ly = 60.0
10
11 Nx = 200
12 Ny = 200
13
14 dx = 0.3
15 dy = 0.3
16
17 dt = 0.1
18 t_final = 10000
19
20 alpha = 5e-3
21
22 T0 = 10.0
23 Q = 0.2
24
25 borehole_x = 30.0
26 borehole_y = 30.0
27
28 # =====
29 # GRID CREATION
30 # =====
31
32 x = np.linspace(0, Lx, Nx)
33 y = np.linspace(0, Ly, Ny)
34
35 X, Y = np.meshgrid(x, y)
36
37 T = np.ones((Ny, Nx)) * T0
38
39 i_b = np.argmin(np.abs(x - borehole_x))
40 j_b = np.argmin(np.abs(y - borehole_y))
41
42 print("Grid shape:", T.shape)
43 print("Borehole index: j =", j_b, ", i =", i_b)
44 print("Borehole grid location: x =", x[i_b], "m, y =", y[j_b], "m")
45
46 # =====
47 # STABILITY CHECK
48 # =====
49
50 Fo_x = alpha * dt / dx**2
51 Fo_y = alpha * dt / dy**2
52 Fo_total = Fo_x + Fo_y
53
54 print("Fo_x =", Fo_x)
55 print("Fo_y =", Fo_y)
56 print("Fo_total =", Fo_total)
57
58 if Fo_total <= 0.5:
59     print("Stable simulation condition satisfied.")
60 else:
61     print("Unstable! Reduce dt or increase dx.")
62
63 # =====
64 # PURE DIFFUSION SIMULATION
```

```

65 # =====
66
67 n_steps = int(t_final / dt)
68
69 print("Total time steps =", n_steps)
70
71 save_times = [2000, 5000, 10000]
72 save_steps = [int(t / dt) for t in save_times]
73
74 saved_results = {}
75
76 for n in range(1, n_steps + 1):
77
78     T_old = T.copy()
79
80     T[1:-1, 1:-1] = (
81         T_old[1:-1, 1:-1]
82         + alpha * dt * (
83             (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx**2
84             +
85             (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy**2
86         )
87     )
88
89     T[j_b, i_b] += Q * dt
90
91     T[0, :] = T[1, :]
92     T[-1, :] = T[-2, :]
93     T[:, 0] = T[:, 1]
94     T[:, -1] = T[:, -2]
95
96     if n in save_steps:
97         saved_results[n * dt] = T.copy()
98         print(f"Saved result at t = {n*dt:.0f} s")
99
100 print("Task 1 pure diffusion simulation finished.")
101
102 # =====
103 # PLOT TEMPERATURE FIELDS
104 # =====
105
106 for time, T_plot in saved_results.items():
107
108     plt.figure(figsize=(8, 6))
109
110     contour = plt.contourf(X, Y, T_plot, levels=50, cmap="hot")
111     plt.colorbar(contour, label="Temperature [deg C]")
112
113     plt.scatter(
114         borehole_x,
115         borehole_y,
116         color="cyan",
117         marker="x",
118         s=100,
119         label="Borehole"
120     )
121
122     plt.xlabel("x [m]")
123     plt.ylabel("y [m]")
124     plt.title(f"Task 1: Pure Diffusion, t = {time:.0f} s")
125     plt.axis("equal")
126     plt.legend()
127     plt.tight_layout()
128     plt.show()
129
130 # =====
131 # ANALYTICAL DIFFUSION LENGTH
132 # =====
133

```

```

134 print("\nAnalytical diffusion length comparison:\n")
135
136 for t in save_times:
137     L = np.sqrt(4 * alpha * t)
138     print(f"t = {t:5d} s --> L(t) = {L:.2f} m")
139
140 # =====
141 # EFFECT OF INCREASING THERMAL DIFFUSIVITY alpha
142 # =====
143
144 alpha_values = [5e-3, 1e-2, 2e-2]
145 comparison_results = {}
146
147 for alpha_test in alpha_values:
148
149     T_alpha = np.ones((Ny, Nx)) * T0
150
151     for n in range(n_steps):
152
153         T_old = T_alpha.copy()
154
155         T_alpha[1:-1, 1:-1] = (
156             T_old[1:-1, 1:-1]
157             + alpha_test * dt * (
158                 (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx**2
159                 +
160                 (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy**2
161             )
162         )
163
164         T_alpha[j_b, i_b] += Q * dt
165
166         T_alpha[0, :] = T_alpha[1, :]
167         T_alpha[-1, :] = T_alpha[-2, :]
168         T_alpha[:, 0] = T_alpha[:, 1]
169         T_alpha[:, -1] = T_alpha[:, -2]
170
171         comparison_results[alpha_test] = T_alpha.copy()
172
173 print("Alpha study completed.")
174
175 # =====
176 # PLOT EFFECT OF THERMAL DIFFUSIVITY
177 # =====
178
179 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
180
181 for ax, alpha_test in zip(axes, alpha_values):
182
183     T_plot = comparison_results[alpha_test]
184
185     contour = ax.contourf(X, Y, T_plot, levels=50, cmap="hot")
186
187     ax.scatter(
188         borehole_x,
189         borehole_y,
190         color="cyan",
191         marker="x",
192         s=80
193     )
194
195     ax.set_title(f"alpha = {alpha_test:.3e}")
196     ax.set_xlabel("x [m]")
197     ax.set_ylabel("y [m]")
198     ax.set_aspect("equal")
199
200 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
201 plt.suptitle("Effect of Increasing Thermal Diffusivity")
202 plt.tight_layout()

```

```

203 plt.show()
204
205 # =====
206 # EFFECT OF SOURCE STRENGTH Q
207 # =====
208
209 Q_values = [0.1, 0.2, 0.4]
210 Q_comparison_results = {}
211
212 for Q_test in Q_values:
213
214     T_Q = np.ones((Ny, Nx)) * T0
215
216     for n in range(n_steps):
217
218         T_old = T_Q.copy()
219
220         T_Q[1:-1, 1:-1] = (
221             T_old[1:-1, 1:-1]
222             + alpha * dt * (
223                 (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx**2
224                 +
225                 (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy**2
226             )
227         )
228
229         T_Q[j_b, i_b] += Q_test * dt
230
231         T_Q[0, :] = T_Q[1, :]
232         T_Q[-1, :] = T_Q[-2, :]
233         T_Q[:, 0] = T_Q[:, 1]
234         T_Q[:, -1] = T_Q[:, -2]
235
236         Q_comparison_results[Q_test] = T_Q.copy()
237
238 print("Q source strength study completed.")
239
240 # =====
241 # PLOT EFFECT OF SOURCE STRENGTH Q
242 # =====
243
244 vmin = min(np.min(Tq) for Tq in Q_comparison_results.values())
245 vmax = max(np.max(Tq) for Tq in Q_comparison_results.values())
246
247 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
248
249 for ax, Q_test in zip(axes, Q_values):
250
251     T_plot = Q_comparison_results[Q_test]
252
253     contour = ax.contourf(
254         X,
255         Y,
256         T_plot,
257         levels=50,
258         cmap="hot",
259         vmin=vmin,
260         vmax=vmax
261     )
262
263     ax.scatter(
264         borehole_x,
265         borehole_y,
266         color="cyan",
267         marker="x",
268         s=80
269     )
270
271     ax.set_title(f"Q = {Q_test:.1f} deg C/s")

```

```

272     ax.set_xlabel("x [m]")
273     ax.set_ylabel("y [m]")
274     ax.set_aspect("equal")
275
276 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
277 plt.suptitle("Effect of Source Strength Q on Pure Diffusion")
278 plt.tight_layout()
279 plt.show()
280
281 # =====
282 # NUMERICAL PLUME RADIUS DETECTION
283 # =====
284
285 threshold = 0.01
286 plume_radius_results = []
287
288 R = np.sqrt((X - borehole_x)**2 + (Y - borehole_y)**2)
289
290 for time, T_plot in saved_results.items():
291
292     delta_T = T_plot - T0
293     plume_mask = delta_T >= threshold
294
295     if np.any(plume_mask):
296         numerical_radius = np.max(R[plume_mask])
297     else:
298         numerical_radius = 0.0
299
300     analytical_radius = np.sqrt(4 * alpha * time)
301
302     plume_radius_results.append({
303         "time_s": time,
304         "numerical_radius_m": numerical_radius,
305         "analytical_L_m": analytical_radius,
306         "difference_m": numerical_radius - analytical_radius
307     })
308
309 for row in plume_radius_results:
310     print(
311         f"t = {row['time_s']:7.0f} s | "
312         f"Numerical radius = {row['numerical_radius_m']:6.2f} m | "
313         f"Analytical L(t) = {row['analytical_L_m']:6.2f} m | "
314         f"Difference = {row['difference_m']:6.2f} m"
315     )
316
317 # =====
318 # GRID REFINEMENT STUDY
319 # =====
320
321 grid_sizes = [100, 200, 400]
322 study_times = [2000, 10000]
323
324 grid_results = {}
325
326 for N in grid_sizes:
327
328     print(f"\nRunning grid {N} x {N}")
329
330     x_g = np.linspace(0, Lx, N)
331     y_g = np.linspace(0, Ly, N)
332
333     X_g, Y_g = np.meshgrid(x_g, y_g)
334
335     dx_g = x_g[1] - x_g[0]
336     dy_g = y_g[1] - y_g[0]
337
338     Fo_total_g = alpha * dt / dx_g**2 + alpha * dt / dy_g**2
339     print("Fo_total =", Fo_total_g)
340

```

```

341     if Fo_total_g > 0.5:
342         print("WARNING: unstable grid/time-step combination!")
343
344     T_g = np.ones((N, N)) * T0
345
346     i_bg = np.argmin(np.abs(x_g - borehole_x))
347     j_bg = np.argmin(np.abs(y_g - borehole_y))
348
349     n_steps_g = int(t_final / dt)
350     save_steps_g = [int(t / dt) for t in study_times]
351
352     grid_results[N] = {}
353
354     for n in range(1, n_steps_g + 1):
355
356         T_old = T_g.copy()
357
358         T_g[1:-1, 1:-1] = (
359             T_old[1:-1, 1:-1]
360             + alpha * dt * (
361                 (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx_g**2
362                 +
363                 (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy_g**2
364             )
365         )
366
367         T_g[j_bg, i_bg] += Q * dt
368
369         T_g[0, :] = T_g[1, :]
370         T_g[-1, :] = T_g[-2, :]
371         T_g[:, 0] = T_g[:, 1]
372         T_g[:, -1] = T_g[:, -2]
373
374         if n in save_steps_g:
375             grid_results[N][n * dt] = {
376                 "X": X_g,
377                 "Y": Y_g,
378                 "T": T_g.copy()
379             }
380
381     print("\nGrid refinement simulation completed.")
382
383     # =====
384     # PLOT GRID REFINEMENT RESULTS
385     # =====
386
387     t_refine = 10000
388
389     vmin = min(np.min(grid_results[N][t_refine]["T"]) for N in grid_sizes)
390     vmax = max(np.max(grid_results[N][t_refine]["T"]) for N in grid_sizes)
391
392     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
393
394     for ax, N in zip(axes, grid_sizes):
395
396         result = grid_results[N][t_refine]
397
398         contour = ax.contourf(
399             result["X"],
400             result["Y"],
401             result["T"],
402             levels=50,
403             cmap="hot",
404             vmin=vmin,
405             vmax=vmax
406         )
407
408         ax.scatter(
409             borehole_x,

```

```

410     borehole_y,
411     color="cyan",
412     marker="x",
413     s=80
414 )
415
416 ax.set_title(f"Grid: {N} x {N}")
417 ax.set_xlabel("x [m]")
418 ax.set_ylabel("y [m]")
419 ax.set_aspect("equal")
420
421 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
422 plt.suptitle(f"Grid Refinement Study at t = {t_refine} s")
423 plt.tight_layout()
424 plt.show()
425
426 # =====
427 # TIME-STEP CONVERGENCE TEST
428 # =====
429
430 dt_values = [0.1, 0.05, 0.02]
431 t_compare = 2000
432
433 timestep_results = {}
434
435 for dt_test in dt_values:
436
437     print(f"\nRunning simulation for dt = {dt_test} s")
438
439     T_dt = np.ones((Ny, Nx)) * T0
440
441     Fo_total_dt = alpha * dt_test / dx**2 + alpha * dt_test / dy**2
442     print("Fo_total =", Fo_total_dt)
443
444     if Fo_total_dt > 0.5:
445         print("WARNING: unstable time step!")
446
447     n_steps_dt = int(t_compare / dt_test)
448
449     for n in range(n_steps_dt):
450
451         T_old = T_dt.copy()
452
453         T_dt[1:-1, 1:-1] = (
454             T_old[1:-1, 1:-1]
455             + alpha * dt_test * (
456                 (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx**2
457                 +
458                 (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy**2
459             )
460         )
461
462         T_dt[j_b, i_b] += Q * dt_test
463
464         T_dt[0, :] = T_dt[1, :]
465         T_dt[-1, :] = T_dt[-2, :]
466         T_dt[:, 0] = T_dt[:, 1]
467         T_dt[:, -1] = T_dt[:, -2]
468
469     timestep_results[dt_test] = T_dt.copy()
470
471 print("\nTime-step convergence test completed.")
472
473 # =====
474 # PLOT TIME-STEP CONVERGENCE
475 # =====
476
477 vmin = min(np.min(Tdt) for Tdt in timestep_results.values())
478 vmax = max(np.max(Tdt) for Tdt in timestep_results.values())

```

```

479 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
480
481
482 for ax, dt_test in zip(axes, dt_values):
483
484     T_plot = timestep_results[dt_test]
485
486     contour = ax.contourf(
487         X,
488         Y,
489         T_plot,
490         levels=50,
491         cmap="hot",
492         vmin=vmin,
493         vmax=vmax
494     )
495
496     ax.scatter(
497         borehole_x,
498         borehole_y,
499         color="cyan",
500         marker="x",
501         s=80
502     )
503
504     ax.set_title(f"dt = {dt_test} s")
505     ax.set_xlabel("x [m]")
506     ax.set_ylabel("y [m]")
507     ax.set_aspect("equal")
508
509 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
510 plt.suptitle(f"Time-Step Convergence Test at t = {t_compare} s")
511 plt.tight_layout()
512 plt.show()
513
514 # =====
515 # ERROR AGAINST FINEST TIME STEP
516 # =====
517
518 T_ref = timestep_results[0.02]
519
520 for dt_test in dt_values:
521
522     T_test = timestep_results[dt_test]
523
524     rmse = np.sqrt(np.mean((T_test - T_ref)**2))
525     max_error = np.max(np.abs(T_test - T_ref))
526
527     print(
528         f"dt = {dt_test:5.2f} s | "
529         f"RMSE = {rmse:.6e} deg C | "
530         f"Max error = {max_error:.6e} deg C"
531     )

```

A.2 Task 2: Pure Advection

Listing 2: Task 2: Pure advection simulation for Group 3

```

1 # =====
2 # TASK 2: PURE ADVECTION
3 # GROUP 3 VELOCITY FIELD
4 # =====
5
6 group = 3
7
8 print("Selected group:", group)
9 print("Using Group 3 velocity field:")
10 print("vx = -0.02(y - 40)")
11 print("vy = 0.02(x - 40)")

```

```

12
13 # =====
14 # BLOCK 1: DEFINE GROUP 3 VELOCITY FIELD
15 # =====
16
17 vx = -0.02 * (Y - 40)
18 vy = 0.02 * (X - 40)
19
20 print("Velocity field created.")
21 print("vx min =", np.min(vx))
22 print("vx max =", np.max(vx))
23 print("vy min =", np.min(vy))
24 print("vy max =", np.max(vy))
25
26 # =====
27 # BLOCK 2: QUIVER PLOT OF VELOCITY FIELD
28 # =====
29
30 plt.figure(figsize=(8, 7))
31
32 skip = 10
33
34 plt.quiver(
35     X[:, ::skip, ::skip],
36     Y[:, ::skip, ::skip],
37     vx[:, ::skip, ::skip],
38     vy[:, ::skip, ::skip],
39     color="blue"
40 )
41
42 plt.scatter(
43     borehole_x,
44     borehole_y,
45     color="red",
46     marker="x",
47     s=120,
48     label="Borehole"
49 )
50
51 plt.xlabel("x [m]")
52 plt.ylabel("y [m]")
53 plt.title("Task 2: Group 3 Velocity Field")
54 plt.axis("equal")
55 plt.legend()
56 plt.tight_layout()
57 plt.show()
58
59 # =====
60 # BLOCK 3: PURE ADVECTION SOLVER
61 # =====
62
63 T_adv = np.ones((Ny, Nx)) * T0
64
65 adv_save_times = [2000, 5000, 10000]
66 adv_save_steps = [int(t / dt) for t in adv_save_times]
67
68 adv_results = {}
69 plume_centers = []
70
71 CFL_x = np.max(np.abs(vx)) * dt / dx
72 CFL_y = np.max(np.abs(vy)) * dt / dy
73 CFL_total = CFL_x + CFL_y
74
75 print("CFL_x =", CFL_x)
76 print("CFL_y =", CFL_y)
77 print("CFL_total =", CFL_total)
78
79 if CFL_total <= 1:
80     print("CFL condition approximately satisfied.")

```

```

81 else:
82     print("WARNING: CFL condition may be unstable. Reduce dt.")
83
84 for n in range(1, n_steps + 1):
85
86     T_old = T_adv.copy()
87
88     dTdx = np.zeros_like(T_adv)
89     dTdy = np.zeros_like(T_adv)
90
91     dTdx[:, 1:-1] = np.where(
92         vx[:, 1:-1] > 0,
93         (T_old[:, 1:-1] - T_old[:, :-2]) / dx,
94         (T_old[:, 2:] - T_old[:, 1:-1]) / dx
95     )
96
97     dTdy[1:-1, :] = np.where(
98         vy[1:-1, :] > 0,
99         (T_old[1:-1, :] - T_old[:-2, :]) / dy,
100        (T_old[2:, :] - T_old[1:-1, :]) / dy
101    )
102
103    T_adv = T_old - dt * (vx * dTdx + vy * dTdy)
104
105    T_adv[j_b, i_b] += Q * dt
106
107    T_adv[0, :] = T_adv[1, :]
108    T_adv[-1, :] = T_adv[-2, :]
109    T_adv[:, 0] = T_adv[:, 1]
110    T_adv[:, -1] = T_adv[:, -2]
111
112    delta_T = T_adv - T0
113    delta_T[delta_T < 0] = 0
114
115    total_heat = np.sum(delta_T)
116
117    if total_heat > 0:
118        xc = np.sum(X * delta_T) / total_heat
119        yc = np.sum(Y * delta_T) / total_heat
120        plume_centers.append((n * dt, xc, yc))
121
122    if n in adv_save_steps:
123        adv_results[n * dt] = T_adv.copy()
124        print(f"Saved advection result at t = {n*dt:.0f} s")
125
126 print("Pure advection simulation finished.")
127
128 # =====
129 # BLOCK 4: PLOT PLUME TRAJECTORY
130 # =====
131
132 plt.figure(figsize=(8, 7))
133
134 final_time = adv_save_times[-1]
135 T_final_adv = adv_results[final_time]
136
137 contour = plt.contourf(
138     X,
139     Y,
140     T_final_adv,
141     levels=50,
142     cmap="hot"
143 )
144
145 plt.colorbar(contour, label="Temperature [deg C]")
146
147 skip = 12
148
149 plt.quiver(

```

```

150     X[:, :skip, ::skip],
151     Y[:, :skip, ::skip],
152     vx[:, :skip, ::skip],
153     vy[:, :skip, ::skip],
154     color="cyan",
155     alpha=0.5
156 )
157
158 plume_centers_array = np.array(plume_centers)
159
160 plt.plot(
161     plume_centers_array[:, 1],
162     plume_centers_array[:, 2],
163     "b-",
164     linewidth=2,
165     label="Plume center trajectory"
166 )
167
168 plt.scatter(
169     plume_centers_array[0, 1],
170     plume_centers_array[0, 2],
171     color="green",
172     s=80,
173     label="Start"
174 )
175
176 plt.scatter(
177     plume_centers_array[-1, 1],
178     plume_centers_array[-1, 2],
179     color="blue",
180     s=80,
181     label="End"
182 )
183
184 plt.scatter(
185     borehole_x,
186     borehole_y,
187     color="red",
188     marker="x",
189     s=120,
190     label="Borehole"
191 )
192
193 plt.xlabel("x [m]")
194 plt.ylabel("y [m]")
195 plt.title("Task 2: Pure Advection Plume Trajectory, Group 3")
196 plt.axis("equal")
197 plt.legend()
198 plt.tight_layout()
199 plt.show()
200
201 # =====
202 # BLOCK 5: PLOT ADVECTION RESULTS AT DIFFERENT TIMES
203 # =====
204
205 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
206
207 for ax, t in zip(axes, adv_save_times):
208
209     contour = ax.contourf(
210         X,
211         Y,
212         adv_results[t],
213         levels=50,
214         cmap="hot"
215     )
216
217     ax.quiver(
218         X[:, :12, ::12],

```

```

219         Y[:, :12, :12],
220         vx[:, :12, :12],
221         vy[:, :12, :12],
222         color="cyan",
223         alpha=0.4
224     )
225
226     ax.scatter(
227         borehole_x,
228         borehole_y,
229         color="red",
230         marker="x",
231         s=100
232     )
233
234     ax.set_title(f"t = {t} s")
235     ax.set_xlabel("x [m]")
236     ax.set_ylabel("y [m]")
237     ax.set_aspect("equal")
238
239 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
240 plt.suptitle("Task 2: Pure Advection Results at Different Times")
241 plt.tight_layout()
242 plt.show()
243
244 # =====
245 # BLOCK 6: TIME STEP EFFECT FOR PURE ADVECTION
246 # =====
247
248 dt_adv_values = [0.1, 0.2, 0.5]
249 t_compare_adv = 2000
250
251 adv_dt_results = {}
252 adv_dt_centers = {}
253
254 for dt_test in dt_adv_values:
255
256     print(f"\nRunning pure advection with dt = {dt_test} s")
257
258     T_test = np.ones((Ny, Nx)) * T0
259     centers = []
260
261     CFL_x = np.max(np.abs(vx)) * dt_test / dx
262     CFL_y = np.max(np.abs(vy)) * dt_test / dy
263     CFL_total = CFL_x + CFL_y
264
265     print("CFL_total =", CFL_total)
266
267     if CFL_total > 1:
268         print("WARNING: CFL condition may be unstable.")
269
270     n_steps_test = int(t_compare_adv / dt_test)
271
272     for n in range(1, n_steps_test + 1):
273
274         T_old = T_test.copy()
275
276         dTdx = np.zeros_like(T_test)
277         dTdy = np.zeros_like(T_test)
278
279         dTdx[:, 1:-1] = np.where(
280             vx[:, 1:-1] > 0,
281             (T_old[:, 1:-1] - T_old[:, :-2]) / dx,
282             (T_old[:, 2:] - T_old[:, 1:-1]) / dx
283         )
284
285         dTdy[1:-1, :] = np.where(
286             vy[1:-1, :] > 0,
287             (T_old[1:-1, :] - T_old[:-2, :]) / dy,

```

```

288         (T_old[2:, :] - T_old[1:-1, :]) / dy
289     )
290
291     T_test = T_old - dt_test * (vx * dTdx + vy * dTdy)
292
293     T_test[j_b, i_b] += Q * dt_test
294
295     T_test[0, :] = T_test[1, :]
296     T_test[-1, :] = T_test[-2, :]
297     T_test[:, 0] = T_test[:, 1]
298     T_test[:, -1] = T_test[:, -2]
299
300     delta_T = T_test - T0
301     delta_T[delta_T < 0] = 0
302
303     total_heat = np.sum(delta_T)
304
305     if total_heat > 0:
306         xc = np.sum(X * delta_T) / total_heat
307         yc = np.sum(Y * delta_T) / total_heat
308         centers.append((n * dt_test, xc, yc))
309
310     adv_dt_results[dt_test] = T_test.copy()
311     adv_dt_centers[dt_test] = np.array(centers)
312
313     print("\nTask 2 time-step effect completed.")
314
315     # =====
316     # BLOCK 7: PLOT TIME STEP EFFECT
317     # =====
318
319     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
320
321     vmin = min(np.min(T_plot) for T_plot in adv_dt_results.values())
322     vmax = max(np.max(T_plot) for T_plot in adv_dt_results.values())
323
324     for ax, dt_test in zip(axes, dt_adv_values):
325
326         T_plot = adv_dt_results[dt_test]
327
328         contour = ax.contourf(
329             X,
330             Y,
331             T_plot,
332             levels=50,
333             cmap="hot",
334             vmin=vmin,
335             vmax=vmax
336         )
337
338         skip = 15
339
340         ax.quiver(
341             X[::skip, ::skip],
342             Y[::skip, ::skip],
343             vx[::skip, ::skip],
344             vy[::skip, ::skip],
345             color="cyan",
346             alpha=0.45
347         )
348
349         centers = adv_dt_centers[dt_test]
350
351         if len(centers) > 0:
352             ax.plot(
353                 centers[:, 1],
354                 centers[:, 2],
355                 "b-",
356                 linewidth=2,

```

```

357         label="Plume trajectory"
358     )
359
360     ax.scatter(
361         borehole_x,
362         borehole_y,
363         color="red",
364         marker="x",
365         s=100
366     )
367
368     ax.set_title(f"Pure Advection, dt = {dt_test} s")
369     ax.set_xlabel("x [m]")
370     ax.set_ylabel("y [m]")
371     ax.set_aspect("equal")
372
373 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
374 plt.suptitle("Effect of Increasing Time Step on Pure Advection")
375 plt.tight_layout()
376 plt.show()
377
378 # =====
379 # BLOCK 8: CFL ANALYSIS
380 # =====
381
382 dt_check_values = [0.1, 0.2, 0.5]
383
384 vmax_x = np.max(np.abs(vx))
385 vmax_y = np.max(np.abs(vy))
386
387 print("Maximum |vx| =", vmax_x, "m/s")
388 print("Maximum |vy| =", vmax_y, "m/s")
389 print()
390
391 for dt_test in dt_check_values:
392
393     CFL_x = vmax_x * dt_test / dx
394     CFL_y = vmax_y * dt_test / dy
395     CFL_total = CFL_x + CFL_y
396
397     print(f"dt = {dt_test:.2f} s")
398     print(f"    CFL_x      = {CFL_x:.4f}")
399     print(f"    CFL_y      = {CFL_y:.4f}")
400     print(f"    CFL_total  = {CFL_total:.4f}")
401
402     if CFL_total <= 1:
403         print("    Status: Stable for explicit upwind advection\n")
404     else:
405         print("    Status: Unstable or risky because CFL_total > 1\n")
406
407 # =====
408 # BLOCK 9: UPWIND VS CENTRAL DIFFERENCE SCHEME
409 # =====
410
411 t_compare = 2000
412
413 schemes = ["upwind", "central"]
414 scheme_results = {}
415
416 for scheme in schemes:
417
418     print(f"\nRunning {scheme} scheme...")
419
420     T_scheme = np.ones((Ny, Nx)) * T0
421
422     n_steps_scheme = int(t_compare / dt)
423
424     for n in range(n_steps_scheme):

```

```

426     T_old = T_scheme.copy()
427
428     dTdx = np.zeros_like(T_scheme)
429     dTdy = np.zeros_like(T_scheme)
430
431     if scheme == "upwind":
432
433         dTdx[:, 1:-1] = np.where(
434             vx[:, 1:-1] > 0,
435             (T_old[:, 1:-1] - T_old[:, :-2]) / dx,
436             (T_old[:, 2:] - T_old[:, 1:-1]) / dx
437         )
438
439         dTdy[1:-1, :] = np.where(
440             vy[1:-1, :] > 0,
441             (T_old[1:-1, :] - T_old[:-2, :]) / dy,
442             (T_old[2:, :] - T_old[1:-1, :]) / dy
443         )
444
445     elif scheme == "central":
446
447         dTdx[:, 1:-1] = (
448             T_old[:, 2:] - T_old[:, :-2]
449         ) / (2 * dx)
450
451         dTdy[1:-1, :] = (
452             T_old[2:, :] - T_old[:-2, :]
453         ) / (2 * dy)
454
455     T_scheme = T_old - dt * (vx * dTdx + vy * dTdy)
456
457     T_scheme[j_b, i_b] += Q * dt
458
459     T_scheme[0, :] = T_scheme[1, :]
460     T_scheme[-1, :] = T_scheme[-2, :]
461     T_scheme[:, 0] = T_scheme[:, 1]
462     T_scheme[:, -1] = T_scheme[:, -2]
463
464     scheme_results[scheme] = T_scheme.copy()
465
466     print("\nScheme comparison completed.")
467
468     # =====
469     # BLOCK 10: PLOT UPWIND VS CENTRAL SCHEME
470     # =====
471
472     fig, axes = plt.subplots(1, 2, figsize=(14, 6))
473
474     clean_results = {}
475
476     for scheme in schemes:
477         clean_results[scheme] = np.nan_to_num(
478             scheme_results[scheme],
479             nan=T0,
480             posinf=T0,
481             neginf=T0
482         )
483
484     vmin = min(np.min(T_plot) for T_plot in clean_results.values())
485     vmax = max(np.max(T_plot) for T_plot in clean_results.values())
486
487     for ax, scheme in zip(axes, schemes):
488
489         T_plot = clean_results[scheme]
490
491         contour = ax.contourf(
492             X,
493             Y,
494             T_plot,

```

```

495         levels=50,
496         cmap="hot",
497         vmin=vmin,
498         vmax=vmax
499     )
500
501     skip = 12
502
503     ax.quiver(
504         X[::skip, ::skip],
505         Y[::skip, ::skip],
506         vx[::skip, ::skip],
507         vy[::skip, ::skip],
508         color="cyan",
509         alpha=0.4
510     )
511
512     ax.scatter(
513         borehole_x,
514         borehole_y,
515         color="red",
516         marker="x",
517         s=100
518     )
519
520     ax.set_title(f"{scheme.capitalize()} Scheme")
521     ax.set_xlabel("x [m]")
522     ax.set_ylabel("y [m]")
523     ax.set_aspect("equal")
524
525 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
526 plt.suptitle("Pure Advection: Upwind vs Central Scheme")
527 plt.tight_layout()
528 plt.show()

```

A.3 Task 3: Advection–Diffusion

Listing 3: Task 3: Advection–diffusion simulation

```

1  # =====
2  # TASK 3: ADVECTION + DIFFUSION
3  # =====
4
5  Fo_x = alpha * dt / dx**2
6  Fo_y = alpha * dt / dy**2
7  Fo_total = Fo_x + Fo_y
8
9  CFL_x = np.max(np.abs(vx)) * dt / dx
10 CFL_y = np.max(np.abs(vy)) * dt / dy
11 CFL_total = CFL_x + CFL_y
12
13 print("----- Diffusion Stability -----")
14 print("Fo_x =", Fo_x)
15 print("Fo_y =", Fo_y)
16 print("Fo_total =", Fo_total)
17
18 print("\n----- Advection Stability -----")
19 print("CFL_x =", CFL_x)
20 print("CFL_y =", CFL_y)
21 print("CFL_total =", CFL_total)
22
23 if Fo_total <= 0.5:
24     print("Diffusion condition satisfied")
25 else:
26     print("WARNING: Diffusion condition not satisfied")
27
28 if CFL_total <= 1:
29     print("Advection condition satisfied")
30 else:

```

```

31     print("WARNING: Advection condition not satisfied")
32
33 # =====
34 # ADVECTION + DIFFUSION SOLVER
35 # =====
36
37 T_ad = np.ones((Ny, Nx)) * T0
38
39 ad_save_times = [2000, 5000, 10000]
40 ad_save_steps = [int(t / dt) for t in ad_save_times]
41
42 ad_results = {}
43 ad_plume_centers = []
44
45 for n in range(1, n_steps + 1):
46
47     T_old = T_ad.copy()
48
49     dTdx = np.zeros_like(T_ad)
50     dTdy = np.zeros_like(T_ad)
51
52     dTdx[:, 1:-1] = np.where(
53         vx[:, 1:-1] > 0,
54         (T_old[:, 1:-1] - T_old[:, :-2]) / dx,
55         (T_old[:, 2:] - T_old[:, 1:-1]) / dx
56     )
57
58     dTdy[1:-1, :] = np.where(
59         vy[1:-1, :] > 0,
60         (T_old[1:-1, :] - T_old[:-2, :]) / dy,
61         (T_old[2:, :] - T_old[1:-1, :]) / dy
62     )
63
64     laplacian = np.zeros_like(T_ad)
65
66     laplacian[1:-1, 1:-1] = (
67         (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx**2
68         +
69         (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy**2
70     )
71
72     T_ad = (
73         T_old
74         - dt * (vx * dTdx + vy * dTdy)
75         + alpha * dt * laplacian
76     )
77
78     T_ad[j_b, i_b] += Q * dt
79
80     T_ad[0, :] = T_ad[1, :]
81     T_ad[-1, :] = T_ad[-2, :]
82     T_ad[:, 0] = T_ad[:, 1]
83     T_ad[:, -1] = T_ad[:, -2]
84
85     delta_T = T_ad - T0
86     delta_T[delta_T < 0] = 0
87
88     total_heat = np.sum(delta_T)
89
90     if total_heat > 0:
91         xc = np.sum(X * delta_T) / total_heat
92         yc = np.sum(Y * delta_T) / total_heat
93         ad_plume_centers.append((n * dt, xc, yc))
94
95     if n in ad_save_steps:
96         ad_results[n * dt] = T_ad.copy()
97         print(f"Saved Task 3 result at t = {n*dt:.0f} s")
98
99 print("Task 3 advection-diffusion simulation finished.")

```

```

100
101 # =====
102 # PLOT ADVECTION + DIFFUSION RESULTS
103 # =====
104
105 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
106
107 vmin = min(np.min(T_plot) for T_plot in ad_results.values())
108 vmax = max(np.max(T_plot) for T_plot in ad_results.values())
109
110 for ax, t in zip(axes, ad_save_times):
111
112     T_plot = ad_results[t]
113
114     contour = ax.contourf(
115         X,
116         Y,
117         T_plot,
118         levels=50,
119         cmap="hot",
120         vmin=vmin,
121         vmax=vmax
122     )
123
124     skip = 12
125
126     ax.quiver(
127         X[::skip, ::skip],
128         Y[::skip, ::skip],
129         vx[::skip, ::skip],
130         vy[::skip, ::skip],
131         color="cyan",
132         alpha=0.4
133     )
134
135     ax.scatter(
136         borehole_x,
137         borehole_y,
138         color="red",
139         marker="x",
140         s=100
141     )
142
143     ax.set_title(f"t = {t} s")
144     ax.set_xlabel("x [m]")
145     ax.set_ylabel("y [m]")
146     ax.set_aspect("equal")
147
148 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
149 plt.suptitle("Task 3: Advection + Diffusion")
150 plt.tight_layout()
151 plt.show()
152
153 # =====
154 # PLOT PLUME TRAJECTORY
155 # =====
156
157 plt.figure(figsize=(8, 7))
158
159 T_final = ad_results[10000]
160
161 contour = plt.contourf(
162     X,
163     Y,
164     T_final,
165     levels=50,
166     cmap="hot"
167 )
168

```

```

169 plt.colorbar(contour, label="Temperature [deg C]")
170
171 skip = 12
172
173 plt.quiver(
174     X[::skip, ::skip],
175     Y[::skip, ::skip],
176     vx[::skip, ::skip],
177     vy[::skip, ::skip],
178     color="cyan",
179     alpha=0.45
180 )
181
182 trajectory = np.array(ad_plume_centers)
183
184 if len(trajectory) > 0:
185     plt.plot(
186         trajectory[:, 1],
187         trajectory[:, 2],
188         "b-",
189         linewidth=2,
190         label="Plume center trajectory"
191     )
192
193 plt.scatter(
194     borehole_x,
195     borehole_y,
196     color="red",
197     marker="x",
198     s=120,
199     label="Borehole"
200 )
201
202 plt.xlabel("x [m]")
203 plt.ylabel("y [m]")
204 plt.title("Task 3: Advection + Diffusion Plume Trajectory")
205 plt.axis("equal")
206 plt.legend()
207 plt.tight_layout()
208 plt.show()
209
210 # =====
211 # STREAMLINE VISUALIZATION
212 # =====
213
214 plt.figure(figsize=(8, 7))
215
216 plt.streamplot(
217     X,
218     Y,
219     vx,
220     vy,
221     color="blue",
222     density=1.5,
223     linewidth=1.2
224 )
225
226 plt.scatter(
227     borehole_x,
228     borehole_y,
229     color="red",
230     marker="x",
231     s=120,
232     label="Borehole"
233 )
234
235 plt.xlabel("x [m]")
236 plt.ylabel("y [m]")
237 plt.title("Group 3 Velocity Field: Streamlines")

```

```

238 plt.axis("equal")
239 plt.legend()
240 plt.tight_layout()
241 plt.show()
242
243 # =====
244 # COMPARE TASK 1, TASK 2, TASK 3 AT t = 10000 s
245 # =====
246
247 compare_time = 10000
248
249 T_diffusion = saved_results[compare_time]
250 T_advection = adv_results[compare_time]
251 T_combined = ad_results[compare_time]
252
253 comparison_fields = {
254     "Task 1: Pure Diffusion": T_diffusion,
255     "Task 2: Pure Advection": T_advection,
256     "Task 3: Advection + Diffusion": T_combined
257 }
258
259 vmin = min(np.min(T_plot) for T_plot in comparison_fields.values())
260 vmax = max(np.max(T_plot) for T_plot in comparison_fields.values())
261
262 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
263
264 for ax, item in zip(axes, comparison_fields.items()):
265
266     title, T_plot = item
267
268     contour = ax.contourf(
269         X,
270         Y,
271         T_plot,
272         levels=50,
273         cmap="hot",
274         vmin=vmin,
275         vmax=vmax
276     )
277
278     ax.scatter(
279         borehole_x,
280         borehole_y,
281         color="red",
282         marker="x",
283         s=100
284     )
285
286     if "Advection" in title:
287         ax.quiver(
288             X[:, :12, ::12],
289             Y[:, :12, ::12],
290             vx[:, :12, ::12],
291             vy[:, :12, ::12],
292             color="cyan",
293             alpha=0.4
294         )
295
296     ax.set_title(title)
297     ax.set_xlabel("x [m]")
298     ax.set_ylabel("y [m]")
299     ax.set_aspect("equal")
300
301 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
302 plt.suptitle(f"Comparison at t = {compare_time} s")
303 plt.tight_layout()
304 plt.show()
305
306 # =====

```

```

307 # TIME STEP EFFECT FOR ADVECTION + DIFFUSION
308 # =====
309
310 dt_task3_values = [0.1, 0.2, 0.5]
311 t_compare_task3 = 2000
312
313 task3_dt_results = {}
314 task3_dt_centers = {}
315
316 for dt_test in dt_task3_values:
317
318     print(f"\nRunning Task 3 with dt = {dt_test} s")
319
320     T_test = np.ones((Ny, Nx)) * T0
321
322     Fo_total_test = alpha * dt_test / dx**2 + alpha * dt_test / dy**2
323     CFL_total_test = (
324         np.max(np.abs(vx)) * dt_test / dx
325         + np.max(np.abs(vy)) * dt_test / dy
326     )
327
328     print("Fo_total =", Fo_total_test)
329     print("CFL_total =", CFL_total_test)
330
331     if Fo_total_test <= 0.5 and CFL_total_test <= 1:
332         print("Status: stable")
333     else:
334         print("Status: unstable or risky")
335
336     n_steps_test = int(t_compare_task3 / dt_test)
337     centers = []
338
339     for n in range(1, n_steps_test + 1):
340
341         T_old = T_test.copy()
342
343         dTdx = np.zeros_like(T_test)
344         dTdy = np.zeros_like(T_test)
345
346         dTdx[:, 1:-1] = np.where(
347             vx[:, 1:-1] > 0,
348             (T_old[:, 1:-1] - T_old[:, :-2]) / dx,
349             (T_old[:, 2:] - T_old[:, 1:-1]) / dx
350         )
351
352         dTdy[1:-1, :] = np.where(
353             vy[1:-1, :] > 0,
354             (T_old[1:-1, :] - T_old[:-2, :]) / dy,
355             (T_old[2:, :] - T_old[1:-1, :]) / dy
356         )
357
358         laplacian = np.zeros_like(T_test)
359
360         laplacian[1:-1, 1:-1] = (
361             (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx**2
362             +
363             (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy**2
364         )
365
366         T_test = (
367             T_old
368             - dt_test * (vx * dTdx + vy * dTdy)
369             + alpha * dt_test * laplacian
370         )
371
372         T_test[j_b, i_b] += Q * dt_test
373
374         T_test[0, :] = T_test[1, :]
375         T_test[-1, :] = T_test[-2, :]

```

```

376     T_test[:, 0] = T_test[:, 1]
377     T_test[:, -1] = T_test[:, -2]
378
379     if not np.all(np.isfinite(T_test)):
380         print(f"Simulation became unstable at step {n}")
381         break
382
383     delta_T = T_test - T0
384     delta_T[delta_T < 0] = 0
385
386     total_heat = np.sum(delta_T)
387
388     if total_heat > 0:
389         xc = np.sum(X * delta_T) / total_heat
390         yc = np.sum(Y * delta_T) / total_heat
391         centers.append((n * dt_test, xc, yc))
392
393     task3_dt_results[dt_test] = np.nan_to_num(
394         T_test,
395         nan=T0,
396         posinf=T0,
397         neginf=T0
398     )
399     task3_dt_centers[dt_test] = np.array(centers)
400
401     print("\nTask 3 time-step effect completed.")
402
403     # =====
404     # PLOT TIME STEP EFFECT
405     # =====
406
407     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
408
409     vmin = min(np.min(T_plot) for T_plot in task3_dt_results.values())
410     vmax = max(np.max(T_plot) for T_plot in task3_dt_results.values())
411
412     for ax, dt_test in zip(axes, dt_task3_values):
413
414         T_plot = task3_dt_results[dt_test]
415
416         contour = ax.contourf(
417             X,
418             Y,
419             T_plot,
420             levels=50,
421             cmap="hot",
422             vmin=vmin,
423             vmax=vmax
424         )
425
426         skip = 12
427
428         ax.quiver(
429             X[::skip, ::skip],
430             Y[::skip, ::skip],
431             vx[::skip, ::skip],
432             vy[::skip, ::skip],
433             color="cyan",
434             alpha=0.4
435         )
436
437         centers = task3_dt_centers[dt_test]
438
439         if len(centers) > 0:
440             ax.plot(
441                 centers[:, 1],
442                 centers[:, 2],
443                 "b-",
444                 linewidth=2,

```

```

445         label="Plume trajectory"
446     )
447
448     ax.scatter(
449         borehole_x,
450         borehole_y,
451         color="red",
452         marker="x",
453         s=100
454     )
455
456     ax.set_title(f"Task 3, dt = {dt_test} s")
457     ax.set_xlabel("x [m]")
458     ax.set_ylabel("y [m]")
459     ax.set_aspect("equal")
460
461 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
462 plt.suptitle("Effect of Increasing Time Step on Advection-Diffusion")
463 plt.tight_layout()
464 plt.show()
465
466 # =====
467 # PECELET NUMBER ANALYSIS
468 # =====
469
470 velocity_magnitude = np.sqrt(vx**2 + vy**2)
471
472 v_mean = np.mean(velocity_magnitude)
473 v_max = np.max(velocity_magnitude)
474
475 L_char = Lx
476
477 Pe_mean = v_mean * L_char / alpha
478 Pe_max = v_max * L_char / alpha
479
480 print("Characteristic length L =", L_char, "m")
481 print("Mean velocity magnitude =", v_mean, "m/s")
482 print("Maximum velocity magnitude =", v_max, "m/s")
483 print()
484 print("Pe_mean =", Pe_mean)
485 print("Pe_max =", Pe_max)
486
487 if Pe_mean > 1:
488     print("Based on mean velocity: system is advection-dominated.")
489 else:
490     print("Based on mean velocity: system is diffusion-dominated.")
491
492 if Pe_max > 1:
493     print("Based on maximum velocity: system is advection-dominated.")
494 else:
495     print("Based on maximum velocity: system is diffusion-dominated.")
496
497 # =====
498 # THERMAL MASS ANALYSIS
499 # =====
500
501 thermal_mass = []
502
503 for t in ad_save_times:
504
505     T_field = ad_results[t]
506
507     delta_T = T_field - T0
508
509     M = np.sum(delta_T) * dx * dy
510
511     thermal_mass.append(M)
512
513     print(f"t = {t:5d} s --> Thermal Mass M(t) = {M:.6f}")

```

```

514 # =====
515 # PLOT THERMAL MASS EVOLUTION
516 # =====
517
518
519 plt.figure(figsize=(7, 5))
520
521 plt.plot(
522     ad_save_times,
523     thermal_mass,
524     "o-b",
525     linewidth=2,
526     markersize=8
527 )
528
529 plt.xlabel("Time [s]")
530 plt.ylabel("Thermal Mass M(t)")
531 plt.title("Thermal Mass Evolution")
532 plt.grid(True)
533 plt.tight_layout()
534 plt.show()
535
536 # =====
537 # NORMALIZED THERMAL MASS
538 # =====
539
540 thermal_mass_norm = np.array(thermal_mass) / thermal_mass[0]
541
542 plt.figure(figsize=(7, 5))
543
544 plt.plot(
545     ad_save_times,
546     thermal_mass_norm,
547     "s-r",
548     linewidth=2
549 )
550
551 plt.xlabel("Time [s]")
552 plt.ylabel("Normalized Thermal Mass")
553 plt.title("Normalized Thermal Mass Evolution")
554 plt.grid(True)
555 plt.tight_layout()
556 plt.show()

```

A.4 Task 4: Time-Varying Source

Listing 4: Task 4: Time-varying source simulation

```

1 # =====
2 # TASK 4: TIME-VARYING SOURCE
3 # =====
4
5 def Q_time(t):
6     """
7     Group 3 time-varying source.
8     Source is ON for first 200 s of every 1500 s period.
9     """
10    if (t % 1500) < 200:
11        return 0.15
12    else:
13        return 0.0
14
15
16 print("Q_time function created.")
17 print("Q(0)      =", Q_time(0))
18 print("Q(100)     =", Q_time(100))
19 print("Q(300)     =", Q_time(300))
20 print("Q(1600)    =", Q_time(1600))
21

```

```

22 # =====
23 # PLOT Q(t)
24 # =====
25
26 time_array = np.arange(0, t_final + dt, dt)
27 Q_array = np.array([Q_time(t) for t in time_array])
28
29 plt.figure(figsize=(10, 4))
30 plt.plot(time_array, Q_array, linewidth=2)
31 plt.xlabel("Time [s]")
32 plt.ylabel("Q(t) [deg C/s]")
33 plt.title("Group 3 Time-Varying Source Term")
34 plt.grid(True)
35 plt.tight_layout()
36 plt.show()
37
38 # =====
39 # TIME-VARYING SOURCE SIMULATION
40 # =====
41
42 T_tv = np.ones((Ny, Nx)) * T0
43
44 tv_save_times = [2000, 5000, 10000]
45 tv_save_steps = [int(t / dt) for t in tv_save_times]
46
47 tv_results = {}
48 plume_centers_tv = []
49 Q_history = []
50
51 for n in range(1, n_steps + 1):
52
53     current_time = n * dt
54
55     T_old = T_tv.copy()
56
57     Q_now = Q_time(current_time)
58     Q_history.append(Q_now)
59
60     dTdx = np.zeros_like(T_tv)
61     dTdy = np.zeros_like(T_tv)
62
63     dTdx[:, 1:-1] = np.where(
64         vx[:, 1:-1] > 0,
65         (T_old[:, 1:-1] - T_old[:, :-2]) / dx,
66         (T_old[:, 2:] - T_old[:, 1:-1]) / dx
67     )
68
69     dTdy[1:-1, :] = np.where(
70         vy[1:-1, :] > 0,
71         (T_old[1:-1, :] - T_old[:-2, :]) / dy,
72         (T_old[2:, :] - T_old[1:-1, :]) / dy
73     )
74
75     laplacian = np.zeros_like(T_tv)
76
77     laplacian[1:-1, 1:-1] = (
78         (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx**2
79         +
80         (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy**2
81     )
82
83     T_tv = (
84         T_old
85         - dt * (vx * dTdx + vy * dTdy)
86         + alpha * dt * laplacian
87     )
88
89     T_tv[j_b, i_b] += Q_now * dt
90

```

```

91     T_tv[0, :] = T_tv[1, :]
92     T_tv[-1, :] = T_tv[-2, :]
93     T_tv[:, 0] = T_tv[:, 1]
94     T_tv[:, -1] = T_tv[:, -2]
95
96     delta_T = T_tv - T0
97     delta_T[delta_T < 0] = 0
98
99     total_heat = np.sum(delta_T)
100
101     if total_heat > 0:
102         xc = np.sum(X * delta_T) / total_heat
103         yc = np.sum(Y * delta_T) / total_heat
104         plume_centers_tv.append((current_time, xc, yc))
105
106     if n in tv_save_steps:
107         tv_results[current_time] = T_tv.copy()
108         print(f"Saved Task 4 result at t = {current_time:.0f} s")
109
110 print("\nTask 4 simulation finished.")
111
112 # =====
113 # PLOT TEMPORAL EVOLUTION
114 # =====
115
116 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
117
118 vmin = min(np.min(T_plot) for T_plot in tv_results.values())
119 vmax = max(np.max(T_plot) for T_plot in tv_results.values())
120
121 for ax, t in zip(axes, tv_save_times):
122
123     T_plot = tv_results[t]
124
125     contour = ax.contourf(
126         X,
127         Y,
128         T_plot,
129         levels=50,
130         cmap="hot",
131         vmin=vmin,
132         vmax=vmax
133     )
134
135     ax.quiver(
136         X[::12, ::12],
137         Y[::12, ::12],
138         vx[::12, ::12],
139         vy[::12, ::12],
140         color="cyan",
141         alpha=0.4
142     )
143
144     ax.scatter(
145         borehole_x,
146         borehole_y,
147         color="red",
148         marker="x",
149         s=100
150     )
151
152     ax.set_title(f"t = {t} s")
153     ax.set_xlabel("x [m]")
154     ax.set_ylabel("y [m]")
155     ax.set_aspect("equal")
156
157 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
158 plt.suptitle("Task 4: Time-Varying Source Evolution")
159 plt.tight_layout()

```

```

160 plt.show()
161
162 # =====
163 # ANALYSE EFFECT OF Q(t)
164 # =====
165
166 threshold = 0.01
167 analysis_results = []
168
169 for t in tv_save_times:
170
171     T_field = tv_results[t]
172     delta_T = T_field - T0
173
174     plume_mask = delta_T >= threshold
175     plume_area = np.sum(plume_mask) * dx * dy
176
177     Tmax = np.max(T_field)
178
179     positive_heat = np.maximum(delta_T, 0)
180     total_heat = np.sum(positive_heat)
181
182     if total_heat > 0:
183         xc = np.sum(X * positive_heat) / total_heat
184         yc = np.sum(Y * positive_heat) / total_heat
185     else:
186         xc = borehole_x
187         yc = borehole_y
188
189     analysis_results.append({
190         "time": t,
191         "plume_area": plume_area,
192         "Tmax": Tmax,
193         "xc": xc,
194         "yc": yc
195     })
196
197     print(f"\nt = {t} s")
198     print("Plume area =", plume_area, "m^2")
199     print("Peak temperature =", Tmax, "deg C")
200     print("Plume center =", f"({xc:.2f}, {yc:.2f})")
201
202 # =====
203 # PLOT PLUME SIZE EVOLUTION
204 # =====
205
206 times = [r["time"] for r in analysis_results]
207 areas = [r["plume_area"] for r in analysis_results]
208
209 plt.figure(figsize=(7, 4))
210 plt.plot(times, areas, "o-b", linewidth=2)
211 plt.xlabel("Time [s]")
212 plt.ylabel("Plume Area [m^2]")
213 plt.title("Effect of Q(t) on Plume Size")
214 plt.grid(True)
215 plt.tight_layout()
216 plt.show()
217
218 # =====
219 # PLOT PEAK TEMPERATURE
220 # =====
221
222 Tmax_list = [r["Tmax"] for r in analysis_results]
223
224 plt.figure(figsize=(7, 4))
225 plt.plot(times, Tmax_list, "s-r", linewidth=2)
226 plt.xlabel("Time [s]")
227 plt.ylabel("Maximum Temperature [deg C]")
228 plt.title("Effect of Q(t) on Peak Temperature")

```

```

229 plt.grid(True)
230 plt.tight_layout()
231 plt.show()
232
233 # =====
234 # PLOT PLUME CENTER MOVEMENT
235 # =====
236
237 xc_list = [r["xc"] for r in analysis_results]
238 yc_list = [r["yc"] for r in analysis_results]
239
240 plt.figure(figsize=(7, 7))
241 plt.plot(xc_list, yc_list, "o-b", linewidth=2)
242
243 plt.scatter(
244     borehole_x,
245     borehole_y,
246     color="red",
247     marker="x",
248     s=120,
249     label="Borehole"
250 )
251
252 for i, t in enumerate(times):
253     plt.annotate(f"{t}s", (xc_list[i], yc_list[i]))
254
255 plt.xlabel("x [m]")
256 plt.ylabel("y [m]")
257 plt.title("Plume Movement under Time-Varying Q(t)")
258 plt.legend()
259 plt.axis("equal")
260 plt.grid(True)
261 plt.tight_layout()
262 plt.show()
263
264 # =====
265 # TIME STEP EFFECT
266 # =====
267
268 dt_task4_values = [0.1, 0.2, 0.5]
269 t_compare_task4 = 2000
270
271 task4_dt_results = {}
272 task4_dt_centers = {}
273 task4_dt_Qhistory = {}
274
275 for dt_test in dt_task4_values:
276
277     print(f"\nRunning Task 4 with dt = {dt_test} s")
278
279     T_test = np.ones((Ny, Nx)) * T0
280     centers = []
281     q_hist = []
282
283     Fo_total_test = alpha * dt_test / dx**2 + alpha * dt_test / dy**2
284     CFL_total_test = (
285         np.max(np.abs(vx)) * dt_test / dx
286         + np.max(np.abs(vy)) * dt_test / dy
287     )
288
289     print("Fo_total =", Fo_total_test)
290     print("CFL_total =", CFL_total_test)
291
292     if Fo_total_test <= 0.5 and CFL_total_test <= 1:
293         print("Status: stable")
294     else:
295         print("Status: unstable or risky")
296
297     n_steps_test = int(t_compare_task4 / dt_test)

```

```

298
299     for n in range(1, n_steps_test + 1):
300
301         current_time = n * dt_test
302         Q_now = Q_time(current_time)
303         q_hist.append(Q_now)
304
305         T_old = T_test.copy()
306
307         dTdx = np.zeros_like(T_test)
308         dTdy = np.zeros_like(T_test)
309
310         dTdx[:, 1:-1] = np.where(
311             vx[:, 1:-1] > 0,
312             (T_old[:, 1:-1] - T_old[:, :-2]) / dx,
313             (T_old[:, 2:] - T_old[:, 1:-1]) / dx
314         )
315
316         dTdy[1:-1, :] = np.where(
317             vy[1:-1, :] > 0,
318             (T_old[1:-1, :] - T_old[:-2, :]) / dy,
319             (T_old[2:, :] - T_old[1:-1, :]) / dy
320         )
321
322         laplacian = np.zeros_like(T_test)
323
324         laplacian[1:-1, 1:-1] = (
325             (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx**2
326             +
327             (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy**2
328         )
329
330         T_test = (
331             T_old
332             - dt_test * (vx * dTdx + vy * dTdy)
333             + alpha * dt_test * laplacian
334         )
335
336         T_test[j_b, i_b] += Q_now * dt_test
337
338         T_test[0, :] = T_test[1, :]
339         T_test[-1, :] = T_test[-2, :]
340         T_test[:, 0] = T_test[:, 1]
341         T_test[:, -1] = T_test[:, -2]
342
343         if not np.all(np.isfinite(T_test)):
344             print(f"Simulation became unstable at step {n}, t = {current_time:.2f} s")
345             break
346
347         delta_T = T_test - T0
348         delta_T[delta_T < 0] = 0
349
350         total_heat = np.sum(delta_T)
351
352         if total_heat > 0:
353             xc = np.sum(X * delta_T) / total_heat
354             yc = np.sum(Y * delta_T) / total_heat
355             centers.append((current_time, xc, yc))
356
357         task4_dt_results[dt_test] = np.nan_to_num(
358             T_test,
359             nan=T0,
360             posinf=T0,
361             neginf=T0
362         )
363         task4_dt_centers[dt_test] = np.array(centers)
364         task4_dt_Qhistory[dt_test] = np.array(q_hist)
365
366     print("\nTask 4 time-step effect completed.")

```

```

367
368 # =====
369 # PLOT TIME STEP EFFECT
370 # =====
371
372 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
373
374 vmin = min(np.min(T_plot) for T_plot in task4_dt_results.values())
375 vmax = max(np.max(T_plot) for T_plot in task4_dt_results.values())
376
377 for ax, dt_test in zip(axes, dt_task4_values):
378
379     T_plot = task4_dt_results[dt_test]
380
381     contour = ax.contourf(
382         X,
383         Y,
384         T_plot,
385         levels=50,
386         cmap="hot",
387         vmin=vmin,
388         vmax=vmax
389     )
390
391     ax.quiver(
392         X[:, :12],
393         Y[:, :12],
394         vx[:, :12],
395         vy[:, :12],
396         color="cyan",
397         alpha=0.4
398     )
399
400     centers = task4_dt_centers[dt_test]
401
402     if len(centers) > 0:
403         ax.plot(
404             centers[:, 1],
405             centers[:, 2],
406             "b-",
407             linewidth=2,
408             label="Plume trajectory"
409         )
410
411     ax.scatter(
412         borehole_x,
413         borehole_y,
414         color="red",
415         marker="x",
416         s=100
417     )
418
419     ax.set_title(f"Task 4, dt = {dt_test} s")
420     ax.set_xlabel("x [m]")
421     ax.set_ylabel("y [m]")
422     ax.set_aspect("equal")
423
424 fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
425 plt.suptitle("Effect of Increasing Time Step on Time-Varying Source Simulation")
426 plt.tight_layout()
427 plt.show()
428
429 # =====
430 # TRACK PLUME CENTER
431 # =====
432
433 plume_centers_tv_array = np.array(plume_centers_tv)
434
435 if len(plume_centers_tv_array) > 0:

```

```

436     time_center = plume_centers_tv_array[:, 0]
437     xc_tv = plume_centers_tv_array[:, 1]
438     yc_tv = plume_centers_tv_array[:, 2]
439
440
441     print("First plume center:", xc_tv[0], yc_tv[0])
442     print("Final plume center:", xc_tv[-1], yc_tv[-1])
443
444     plt.figure(figsize=(8, 7))
445
446     plt.plot(
447         xc_tv,
448         yc_tv,
449         linewidth=2,
450         label="Plume center trajectory"
451     )
452
453     plt.scatter(
454         borehole_x,
455         borehole_y,
456         color="red",
457         marker="x",
458         s=120,
459         label="Borehole"
460     )
461
462     plt.scatter(
463         xc_tv[0],
464         yc_tv[0],
465         color="green",
466         s=80,
467         label="Start"
468     )
469
470     plt.scatter(
471         xc_tv[-1],
472         yc_tv[-1],
473         color="blue",
474         s=80,
475         label="End"
476     )
477
478     plt.xlabel("x center [m]")
479     plt.ylabel("y center [m]")
480     plt.title("Task 4: Plume Center Trajectory")
481     plt.axis("equal")
482     plt.legend()
483     plt.grid(True)
484     plt.tight_layout()
485     plt.show()
486
487     plt.figure(figsize=(9, 5))
488
489     plt.plot(time_center, xc_tv, label="xc(t)", linewidth=2)
490     plt.plot(time_center, yc_tv, label="yc(t)", linewidth=2)
491
492     plt.xlabel("Time [s]")
493     plt.ylabel("Plume center position [m]")
494     plt.title("Task 4: Plume Center Coordinates over Time")
495     plt.legend()
496     plt.grid(True)
497     plt.tight_layout()
498     plt.show()
499 else:
500     print("No plume center data available.")

```

A.5 Task 5: Multiple Boreholes

Listing 5: Task 5: Four-borehole simulation and interaction analysis

```

1 # =====
2 # TASK 5: FOUR BOREHOLES
3 # =====
4
5 boreholes = [
6     (19.8, 19.8),
7     (39.9, 19.8),
8     (19.8, 39.9),
9     (39.9, 39.9)
10 ]
11
12 borehole_indices = []
13
14 for bx, by in boreholes:
15
16     i_bh = np.argmin(np.abs(x - bx))
17     j_bh = np.argmin(np.abs(y - by))
18
19     borehole_indices.append((j_bh, i_bh))
20
21     print(
22         f"Borehole at ({bx}, {by}) -> grid index (j={j_bh}, i={i_bh}), "
23         f"actual grid location ({x[i_bh]:.2f}, {y[j_bh]:.2f})"
24     )
25
26 # =====
27 # MULTIPLE BOREHOLES SIMULATION
28 # =====
29
30 T_multi = np.ones((Ny, Nx)) * T0
31
32 multi_save_times = [2000, 5000, 10000]
33 multi_save_steps = [int(t / dt) for t in multi_save_times]
34
35 multi_results = {}
36
37 for n in range(1, n_steps + 1):
38
39     current_time = n * dt
40     Q_now = Q_time(current_time)
41
42     T_old = T_multi.copy()
43
44     dTdx = np.zeros_like(T_multi)
45     dTdy = np.zeros_like(T_multi)
46
47     dTdx[:, 1:-1] = np.where(
48         vx[:, 1:-1] > 0,
49         (T_old[:, 1:-1] - T_old[:, :-2]) / dx,
50         (T_old[:, 2:] - T_old[:, 1:-1]) / dx
51     )
52
53     dTdy[1:-1, :] = np.where(
54         vy[1:-1, :] > 0,
55         (T_old[1:-1, :] - T_old[:-2, :]) / dy,
56         (T_old[2:, :] - T_old[1:-1, :]) / dy
57     )
58
59     laplacian = np.zeros_like(T_multi)
60
61     laplacian[1:-1, 1:-1] = (
62         (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx**2
63         +
64         (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy**2
65     )
66
67     T_multi = (
68         T_old

```

```

69         - dt * (vx * dTdx + vy * dTdy)
70         + alpha * dt * laplacian
71     )
72
73     for j_bh, i_bh in borehole_indices:
74         T_multi[j_bh, i_bh] += Q_now * dt
75
76     T_multi[0, :] = T_multi[1, :]
77     T_multi[-1, :] = T_multi[-2, :]
78     T_multi[:, 0] = T_multi[:, 1]
79     T_multi[:, -1] = T_multi[:, -2]
80
81     if n in multi_save_steps:
82         multi_results[current_time] = T_multi.copy()
83         print(f"Saved Task 5 result at t = {current_time:.0f} s")
84
85     print("\nTask 5 multi-borehole simulation finished.")
86
87     # =====
88     # PLOT MULTI-BOREHOLE TEMPERATURE FIELD
89     # =====
90
91     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
92
93     vmin = min(np.min(T_plot) for T_plot in multi_results.values())
94     vmax = max(np.max(T_plot) for T_plot in multi_results.values())
95
96     for ax, t in zip(axes, multi_save_times):
97
98         T_plot = multi_results[t]
99
100        contour = ax.contourf(
101            X,
102            Y,
103            T_plot,
104            levels=50,
105            cmap="hot",
106            vmin=vmin,
107            vmax=vmax
108        )
109
110        ax.quiver(
111            X[::12, ::12],
112            Y[::12, ::12],
113            vx[::12, ::12],
114            vy[::12, ::12],
115            color="cyan",
116            alpha=0.4
117        )
118
119        for bx, by in boreholes:
120            ax.scatter(
121                bx,
122                by,
123                color="red",
124                marker="x",
125                s=100
126            )
127
128        ax.set_title(f"t = {t} s")
129        ax.set_xlabel("x [m]")
130        ax.set_ylabel("y [m]")
131        ax.set_aspect("equal")
132
133     fig.colorbar(contour, ax=axes.ravel().tolist(), label="Temperature [deg C]")
134     plt.suptitle("Task 5: Four Boreholes Temperature Field and Interference")
135     plt.tight_layout()
136     plt.show()
137

```

```

138 # =====
139 # BOREHOLE INTERACTION ANALYSIS
140 # =====
141
142 T_final_multi = multi_results[10000]
143
144 interaction_results = []
145
146 for i in range(len(boreholes)):
147     for j in range(i + 1, len(boreholes)):
148
149         x1, y1 = boreholes[i]
150         x2, y2 = boreholes[j]
151
152         xm = (x1 + x2) / 2
153         ym = (y1 + y2) / 2
154
155         im = np.argmin(np.abs(x - xm))
156         jm = np.argmin(np.abs(y - ym))
157
158         T_mid = T_final_multi[jm, im]
159         distance = np.sqrt((x2 - x1)**2 + (y2 - y1)**2)
160
161         interaction_results.append({
162             "pair": f"B{i+1}-B{j+1}",
163             "distance_m": distance,
164             "midpoint_x": xm,
165             "midpoint_y": ym,
166             "T_mid": T_mid,
167             "delta_T_mid": T_mid - T0
168         })
169
170 for r in interaction_results:
171     print(
172         f"{r['pair']} | distance = {r['distance_m']:.2f} m | "
173         f"midpoint = ({r['midpoint_x']:.2f}, {r['midpoint_y']:.2f}) | "
174         f"T_mid = {r['T_mid']:.5f} deg C | "
175         f"dT_mid = {r['delta_T_mid']:.5f} deg C"
176     )
177
178 # =====
179 # PLOT INTERACTION MIDPOINTS
180 # =====
181
182 plt.figure(figsize=(8, 7))
183
184 contour = plt.contourf(
185     X,
186     Y,
187     T_final_multi,
188     levels=50,
189     cmap="hot"
190 )
191
192 plt.colorbar(contour, label="Temperature [deg C]")
193
194 plt.quiver(
195     X[::12, ::12],
196     Y[::12, ::12],
197     vx[::12, ::12],
198     vy[::12, ::12],
199     color="cyan",
200     alpha=0.4
201 )
202
203 for idx, (bx, by) in enumerate(boreholes):
204     plt.scatter(bx, by, color="red", marker="x", s=120)
205     plt.text(bx + 0.7, by + 0.7, f"B{idx+1}", color="white", fontsize=11)
206

```

```

207 for r in interaction_results:
208     plt.scatter(r["midpoint_x"], r["midpoint_y"], color="blue", s=60)
209     plt.text(
210         r["midpoint_x"] + 0.5,
211         r["midpoint_y"] + 0.5,
212         r["pair"],
213         color="blue",
214         fontsize=10
215     )
216
217 plt.xlabel("x [m]")
218 plt.ylabel("y [m]")
219 plt.title("Task 5: Borehole Interaction Midpoints at t = 10000 s")
220 plt.axis("equal")
221 plt.tight_layout()
222 plt.show()
223
224 # =====
225 # SUPERPOSITION TEST
226 # =====
227
228 single_borehole_results = {}
229
230 for bh_id, item in enumerate(borehole_indices, start=1):
231
232     j_single, i_single = item
233
234     print(f"\nRunning single-borehole simulation for B{bh_id}...")
235
236     T_single = np.ones((Ny, Nx)) * T0
237
238     for n in range(1, n_steps + 1):
239
240         current_time = n * dt
241         Q_now = Q_time(current_time)
242
243         T_old = T_single.copy()
244
245         dTdx = np.zeros_like(T_single)
246         dTdy = np.zeros_like(T_single)
247
248         dTdx[:, 1:-1] = np.where(
249             vx[:, 1:-1] > 0,
250             (T_old[:, 1:-1] - T_old[:, :-2]) / dx,
251             (T_old[:, 2:] - T_old[:, 1:-1]) / dx
252         )
253
254         dTdy[1:-1, :] = np.where(
255             vy[1:-1, :] > 0,
256             (T_old[1:-1, :] - T_old[:-2, :]) / dy,
257             (T_old[2:, :] - T_old[1:-1, :]) / dy
258         )
259
260         laplacian = np.zeros_like(T_single)
261
262         laplacian[1:-1, 1:-1] = (
263             (T_old[1:-1, 2:] - 2*T_old[1:-1, 1:-1] + T_old[1:-1, :-2]) / dx**2
264             +
265             (T_old[2:, 1:-1] - 2*T_old[1:-1, 1:-1] + T_old[:-2, 1:-1]) / dy**2
266         )
267
268         T_single = (
269             T_old
270             - dt * (vx * dTdx + vy * dTdy)
271             + alpha * dt * laplacian
272         )
273
274         T_single[j_single, i_single] += Q_now * dt
275

```

```

276     T_single[0, :] = T_single[1, :]
277     T_single[-1, :] = T_single[-2, :]
278     T_single[:, 0] = T_single[:, 1]
279     T_single[:, -1] = T_single[:, -2]
280
281     single_borehole_results[bh_id] = T_single.copy()
282
283 print("\nAll four single-borehole simulations completed.")
284
285 # =====
286 # BUILD SUPERPOSITION FIELD
287 # =====
288
289 T_superposition = np.ones((Ny, Nx)) * T0
290
291 for bh_id in single_borehole_results:
292     T_superposition += single_borehole_results[bh_id] - T0
293
294 T_multiple = multi_results[10000]
295
296 difference_field = T_multiple - T_superposition
297
298 rmse_superposition = np.sqrt(np.mean(difference_field**2))
299 max_error_superposition = np.max(np.abs(difference_field))
300
301 print("Superposition RMSE =", rmse_superposition)
302 print("Superposition max absolute error =", max_error_superposition)
303
304 # =====
305 # PLOT SUPERPOSITION TEST
306 # =====
307
308 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
309
310 fields = [
311     ("Multiple-borehole simulation", T_multiple),
312     ("Sum of four single-borehole simulations", T_superposition),
313     ("Difference field", difference_field)
314 ]
315
316 vmin_main = min(np.min(T_multiple), np.min(T_superposition))
317 vmax_main = max(np.max(T_multiple), np.max(T_superposition))
318
319 diff_abs = np.max(np.abs(difference_field))
320
321 for ax, item in zip(axes, fields):
322
323     title, field = item
324
325     if title == "Difference field":
326
327         contour = ax.contourf(
328             X,
329             Y,
330             field,
331             levels=50,
332             cmap="coolwarm",
333             vmin=-diff_abs,
334             vmax=diff_abs
335         )
336
337     else:
338
339         contour = ax.contourf(
340             X,
341             Y,
342             field,
343             levels=50,
344             cmap="hot",

```

```

345         vmin=vmin_main,
346         vmax=vmax_main
347     )
348
349     for bx, by in boreholes:
350         ax.scatter(
351             bx,
352             by,
353             color="red",
354             marker="x",
355             s=100
356         )
357
358     ax.set_title(title)
359     ax.set_xlabel("x [m]")
360     ax.set_ylabel("y [m]")
361     ax.set_aspect("equal")
362
363 fig.colorbar(contour, ax=axes.ravel().tolist())
364 plt.suptitle("Task 5: Superposition Test")
365 plt.tight_layout()
366 plt.show()

```